

ORACLE
NetSuite

Custom GL Lines Plug-in



2025.2

September 3, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Custom GL Lines Plug-in Overview	1
Custom GL Lines Plug-in Process Flow	2
Transaction Types Supported By Custom GL Lines Plug-in	4
Custom GL Lines Plug-in Implementation Development	6
Gather Required Internal IDs	8
Plug-in Implementation Script File Creation for SuiteScript 1.0	8
Plug-in Implementation Script File Creation for SuiteScript 2.0	10
Adding the Plug-in Implementation	12
Test the Plug-in Implementation	13
Bundle the Plug-in Implementation	14
Custom GL Lines Plug-in Implementation Administration	14
Enabling Features for a Custom GL Lines Plug-in Implementation	15
Installing a Custom GL Lines Plug-in Bundle	15
Configuring the Custom GL Lines Plug-in Implementation	16
Enabling the Custom GL Lines Plug-in Implementation	17
Custom GL Lines Plug-in Implementation System Notes	18
Custom GL Lines Plug-In Interface Definition	20
customizeGllImpact(transactionRecord, standardLines, customLines, book)	22
StandardLines	22
CustomLines	30
AccountingBook	41
Record	43
Common StandardLine and CustomLine Object Methods	44
customizeGllImpact(context)	50
AccountingBook	51
CustomLines	52
ReadOnlyTransactionRecord	58
StandardLines	65
Custom GL Lines Plug-in Reference	72
Custom GL Lines Plug-in Example Code	72
Utility Files for a Custom GL Lines Plug-In Implementation	73
Using a Custom Record to Reference Internal NetSuite IDs Example	74
Classifications	76
Classification Handling on Standard Transactions	76
Classification Handling on Journals	77
Classification Handling on Custom Transactions	78
Multiple Currencies Examples	78
Find the Base Currency	79
Base Currency from Accounting Book Record	79
Transaction-Specific Exchange Rates	80
Convert Currencies with SuiteScript	81
Custom GL Lines Plug-in Guidelines and Best Practices	81
Error Messages for Custom GL Lines Plug-in	83
Invalid Operation Error Messages	83
General Output Validation Error Messages	84
Validation Error Messages for Invalid Values	85
Validation Error Messages for Empty Values	86
Custom Lines on Reports and in Searches	87
Custom GL Lines on NetSuite Reports	87
Searches for Custom GL Lines	88
Custom GL Lines Audit Log	89
Accessing the Custom GL Lines Audit Log	89
Custom GL Lines Audit Log Page	90

Reviewing Custom GL Lines Plug-in Executions	94
Deleting a Custom GL Lines Plug-in Implementation	95

Custom GL Lines Plug-in Overview

The Custom GL Lines Plug-in changes the general ledger impact of standard and custom transactions. Use the Custom GL Lines Plug-in to comply with a wide range of global accounting standards by applying custom transaction logic that adds lines to the NetSuite GL Impact page.

The Custom GL Lines Plug-in can set and read custom segment values from custom and standard lines, as well as from transaction records. By default, column segment values are sourced from the body. The Custom GL Lines Plug-in can change this value, even for segments that are applied only to the body.

Solution providers can create implementations of the plug-in to add GL lines to transactions. These plug-in implementations can be bundled and distributed as SuiteApps. NetSuite account administrators can install the SuiteApp and configure custom GL plug-in implementations for specific transaction types, subsidiaries, and accounting books. OneWorld accounts can set a plug-in implementation to run for specific subsidiaries, and accounts with the Multi-Book Accounting feature enabled can set a plug-in implementation to run for specific accounting books.

For example, a solution provider may create a plug-in implementation to capture locale-specific GL impact on standard transaction types. Plug-in implementations could also add bank, credit card, and wire charges to payments, cash sales, and refunds, or create unique discount or markup rules that add an amount to invoices or bills.

The following screenshot shows the GL Impact page for an invoice with custom lines created by the Tax Expense On Accruals plug-in implementation:

GL Impact Return to Invoice #62														
Accounting Book: Primary Accounting Book Total: 6														
Accounting Book	Account	Amount (Debit)	Amount (Credit)	Base Currency	Posting	Memo	Name	Subsidiary	Department	Class	Location	Custom Script	Rev Rec test XX	Product line
Primary Accounting Book	1100 Accounts Receivable	\$200.00		US Dollar	Yes		Acme Sys Inc.	Parent Company			East Coast			
Primary Accounting Book	150000 Deferred Revenue		\$100.00	US Dollar	Yes		Acme Sys Inc.	Parent Company			East Coast			
Primary Accounting Book	4000 Sales		\$50.00	US Dollar	Yes		Acme Sys Inc.	Parent Company			East Coast			
Primary Accounting Book	5000 Purchases	\$25.00		US Dollar	Yes	Cost of Sales	Acme Sys Inc.	Parent Company			East Coast	Custom GL test		
Primary Accounting Book	1310 Prepaid Income Taxes		\$25.00	US Dollar	Yes	Cost of Sales	Acme Sys Inc.	Parent Company			East Coast	Custom GL test		
Primary Accounting Book	4000 Sales		\$50.00	US Dollar	Yes		Acme Sys Inc.	Parent Company			East Coast			

In the preceding screenshot, the **Custom Script** column links to the plug-in implementation that created the custom line. Since custom lines created by a plug-in implementation are linked to the original transaction, the custom lines can appear in reports and searches. See [Custom Lines on Reports and in Searches](#).

For information about the Custom GL Lines plug-in, see the following topics:

NetSuite Role	For more information, see ...
All roles	Custom GL Lines Plug-in Process Flow Transaction Types Supported By Custom GL Lines Plug-in
Developer	Custom GL Lines Plug-in Implementation Development
Administrator	Custom GL Lines Plug-in Implementation Administration

**Important:** The Custom GL Lines plug-in also supports SuiteScript 2.x. For more information about the functions to use for SuiteScript 2.x, see [Plug-in Implementation Script File Creation for SuiteScript 2.0](#) and `customizeGLImpact(context)`.

Custom GL Lines Plug-in Process Flow

A Custom GL Lines plug-in implementation runs based on how the plug-in implementation it's configured. The implementation runs when you save a transaction of the type and subsidiary you've configured. The subsidiary is the subsidiary to which the transaction belongs. The plug-in implementation may also run multiple times, one time for each accounting book configured for the plug-in implementation.

For example, the following plug-in implementation runs when you save an invoice for the Parent Company or Czech Subsidiary, and it runs on both the primary and secondary books.

Custom GL Lines: Configuration

Save

Cancel

Name

Cash Basis on Invoice Payment

ID

customscript_cash_basis_impl

Status

Testing

Log Level

Debug

☐ Asynchronous

☐ Run in Closed Periods

Effective Date

End Date

Execution Rules

System Notes

Transaction Type

Inventory Transfer

Inventory Worksheet

Invoice

☐ All

Save

Cancel

Subsidiaries

Parent Company

Parent Company : Canada subsidiary

Parent Company : Czech Sub

☐ All

Accounting Books

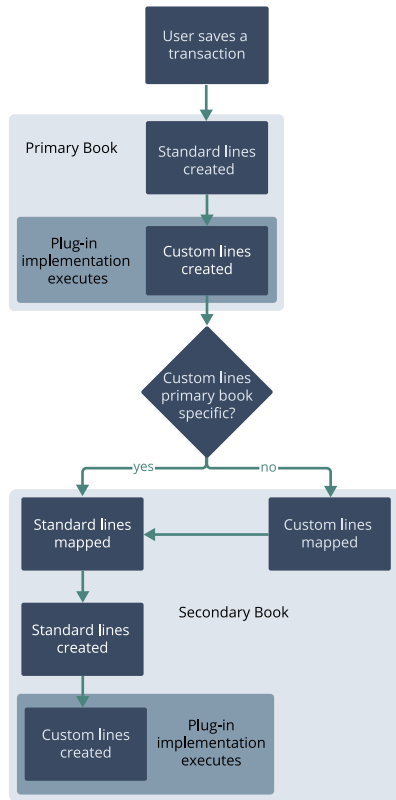
Primary Accounting Book

German Accounting Book

Czech Accounting Book

☐ All

The following diagram shows the processing model of the Custom GL Lines plug-in implementation:



The plug-in implementation executes according to the following process:

1. User saves a transaction for an entity in a subsidiary. If the transaction type, entity, and subsidiary match the plug-in implementation configuration, the plug-in runs on that transaction. The plug-in can run either during transaction the save or asynchronously, depending on the Asynchronous box on the configuration page The subsidiary is the subsidiary to which the transaction belongs. NetSuite first creates the standard lines for the primary book.
2. Plug-in implementation runs on the primary accounting book. NetSuite calls the [customizeGImpact\(transactionRecord, standardLines, customLines, book\)](#) or [customizeGImpact\(context\)](#) functions with the primary book [AccountingBook](#) object. NetSuite creates any custom lines in the primary book set up for the plug-in implementation. If you don't use Multi-Book Accounting, the process is complete. Otherwise, NetSuite continues to map standard and custom lines to the secondary books.
3. NetSuite determines which lines should be applied to any secondary books. By default, NetSuite maps standard lines to the appropriate secondary accounting books, based on any mapping rules set up for Multi-Book Accounting. NetSuite does the same mapping for all custom lines with the [setBookSpecific\(bookSpecific\)](#) or [isBookSpecific\(\)](#) methods set to false.
4. NetSuite creates standard lines, plus any mapped standard lines and any custom lines that aren't specific to the primary book, in the secondary accounting books.
5. The plug-in implementation runs on the secondary accounting books configured for the plug-in implementation. NetSuite executes the [customizeGImpact\(transactionRecord, standardLines, customLines, book\)](#) function for each secondary book [AccountingBook](#) object for the transaction.

NetSuite creates the custom lines on the secondary accounting book.

6. Transaction save finishes. You can see the general ledger impact of the transaction by going to Actions > GL Impact. If the plug-in runs asynchronously and is still pending, an information message appears on the GL Impact page. Custom GL Lines from asynchronous plug-ins are shown after the plug-in runs. When costing updates are performed (for example, the COGS update), the plug-in may run asynchronously, and the values on the GL Impact page may change after the update.

 **Important:** The current version of the Custom GL Lines Plug-in is always used.

Transaction Types Supported By Custom GL Lines Plug-in

The following table lists the transactions supported by the Custom GL Lines Plug-in.

Transaction Name	ID
Assembly Build	assemblybuild
Assembly Unbuild	assemblyunbuild
Bill	vendorbill
Bill Credit	vendorcredit
Bill Payment	vendorpayment
Blanket Purchase Order	blanketpurchaseorder
Book-Specific Journal Entry**	journalentry
Cash Refund	cashrefund
Cash Sale	cashsale
Check	check
Credit Card Charge	creditcardcharge
Credit Card Refund	creditcardrefund
Credit Memo	creditmemo
Customer Deposit	customerdeposit
Customer Payment	customerpayment
Customer Refund	customerrefund
Deposit	deposit
Deposit Application	depositapplication
Expense Report	expensereport
Finance Charge *	invoice

Transaction Name	ID
Inventory Adjustment	inventoryadjustment
Inventory Cost Revaluation	inventorycostrevaluation
Inventory Transfer	inventorytransfer
Inventory Worksheet	inventoryworksheet
Invoice *	invoice
Item Fulfillment	itemfulfillment
Item Receipt	itemreceipt
Journal *	journalentry
Ownership Transfer	ownershiptransfer
Opportunity	opportunity
Paycheck Journal	paycheckjournal
Purchase Contract	purchasecontract
Purchase Order	purchaseorder
Return Authorization	returnauthorization
Revenue Commitment *	revenuecommitment
Revenue Commitment Reversal	revenuecommitmentreversal
Sales Order *	salesorder
Store Pickup Fulfillment	storepickupfulfillment
Vendor Prepayment	vendorprepayment
Vendor Return Authorization	vendorreturnauthorization
Work Order	workorder
Work Order Close	workorderclose
Work Order Completion	workordercompletion
Work Order Issue	workorderissue
* See Transaction Support Exceptions. ** Book-specific journal entries use the same transaction type ID as a standard journal entry. You can distinguish book-specific entries from standard ones by checking the accountingbook field on the TransactionRecord object (see Custom GL Lines Plug-In Interface Definition). Standard journal entries will have a null value in this field, while book-specific journal entries return the NetSuite internal ID of the accounting book. No matter how your accounting books are set up, the Custom GL Lines Plug-in runs only one time for the accounting book of the journal entry. If the plug-in is not configured for the accounting book containing the journal entry, it won't run at all.	

Transaction Support Exceptions

The following transactions are supported with restrictions:

- **Finance Charge.** A special type of invoice, the plug-in supports this transaction type when created through an invoice form.
- **Journal.** The plug-in supports Journals except in the following cases:
 - Voiding journal and reversing journal
These journals aren't supported because they cancel the original transaction and shouldn't create new GL impact
 - Open balance journal
 - Statistical journal
 - Intercompany journal
 - Intercompany elimination journal
 - Revaluation journals
 - Collect Tegata
 - Pay Tegata
 - Revenue reclassification
 - Revenue arrangements
 - Time posting to journal entry
 - Recognize gift certificate income
 - Allocate paycheck to jobs
 - Payroll: Create funding transaction
 - Payroll: YTD process
 - Payroll batch journal
 - Paycheck journal
- **Revenue Commitment.** A plug-in implementation doesn't run on book-specific revenue commitments.
- **Sales Order.** A plug-in implementation doesn't run on book-specific sales orders.

Transaction Processing

An implementation of the plug-in processes transactions by the following methods:

- **Voided transactions.** Voided transactions with custom lines are handled the same as voided transaction with standard GL impact lines. See the help topic [Voiding, Deleting, or Closing Transactions](#).
- **Memorized transactions.** When you enter the transaction, the plug-in implementation creates custom lines. The transaction is also set to Posting or Non-Posting, matching the value for the standard lines.
- **Due date.** Due date is set on custom lines when you save the transaction, and it matches the transaction date. This lets the custom line appear on aging reports.

Custom GL Lines Plug-in Implementation Development

You can develop a Custom GL Lines plug-in implementation for each use case where you want to change general ledger entries for transactions. One plug-in implementation can process multiple transaction types, or you can create a separate plug-in implementation for each transaction type.

To create a plug-in implementation, use a development account to build and test it. Then, use SuiteBundler to bundle the plug-in objects and distribute them to other NetSuite accounts. NetSuite administrators use the bundle to install and enable the plug-in implementation. For more information about administration tasks for a Custom GL Lines Plug-in bundle, see [Custom GL Lines Plug-in Implementation Administration](#).

Note: If you use the Custom GL Lines Plug-in with Custom Segments, any segments affected by the implementation must be flagged as having GL impact. When the plug-in uses a custom segment that is not marked as having a GL impact, you'll see an error message.

What you include in a Custom GL Lines plug-in implementation depends on the accounting features used in the NetSuite account and the following properties that you configure for each plug-in implementation:

- **Transaction types.** A Custom GL Lines plug-in implementation runs when you save a transaction it's set up for, where the entity for the transaction is associated with a specific subsidiary. You can include code in the script file that applies only to specific transaction types.
- **Subsidiaries.** In a One World account, the plug-in implementation affects only the general ledger entries for the subsidiaries you set up for it.
- **Accounting books.** The plug-in implementation affects only the accounting books you set up for it. If you use Multi-Book Accounting, you can have the plug-in implementation change or add custom general ledger entries in multiple accounting books. If you do not have the Multi-Book Accounting feature enabled, the plug-in implementation affects only the general ledger entries in the primary accounting book.

Note: You can have more than one Custom GL Lines plug-in implementation enabled in an account. If you enable a plug-in implementation that conflicts with an existing implementation, NetSuite warns you that having several implementations for one transaction can cause longer save times. You can save the implementation, but if you notice long transaction save times, try setting one or more plug-in implementations to run asynchronously.

For more information about how a Custom GL Lines plug-in implementation modifies the general ledger impact for a transaction, see [Custom GL Lines Plug-in Process Flow](#).

The following table describes the basic steps to develop a single Custom GL Lines plug-in implementation:

Step	Description
Enable features	Enable the Custom GL Lines Plug-in and Server SuiteScript features. See Enabling Features for a Custom GL Lines Plug-in Implementation .
Gather required internal IDs	In a plug-in implementation, you can reference internal NetSuite IDs for records and accounts. Before you create the script file, make sure you note these values. See Gather Required Internal IDs .
Create script file	Create the script file for your Custom GL Lines plug-in implementation. See Plug-in Implementation Script File Creation for SuiteScript 1.0 .
Add the plug-in implementation	Add the plug-in implementation using the plug-in script file and any required utility files that you created in the previous step to the development account. See Adding the Plug-in Implementation .
Test the plug-in implementation	Test how the plug-in implementation works. See Test the Plug-in Implementation .
Bundle the plug-in implementation	Bundle both the plug-in implementation and other required objects for distribution to other NetSuite accounts.

Step	Description
	See Bundle the Plug-in Implementation .

Gather Required Internal IDs

When you create a Custom GL Lines plug-in implementation, you may refer to NetSuite records by their internal IDs.

The following table describes the NetSuite internal IDs you'll need if you want to use those records in the plug-in script file:

Record Type	Location of ID
Accounts	Setup > Accounting > Chart of Accounts
Accounting books	Setup > Accounting > Accounting Books
Departments	Setup > Company > Departments
Classes	Setup > Company > Classes
Locations	Setup > Company > Locations
Subsidiaries	Setup > Company > Subsidiaries
Tax codes	Setup > Accounting > Tax Codes
Tax types	Setup > Accounting > Tax Types



Note: You can use helper functions in a utility file so you don't have to include internal NetSuite IDs directly in your plug-in logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Plug-in Implementation Script File Creation for SuiteScript 1.0

You can use the SuiteCloud IDE, or any JavaScript IDE or text editor, to create the JavaScript file with your business logic for the Custom GL Lines plug-in implementation.

The following table describes the functions and objects available in a Custom GL Lines plug-in implementation script file:

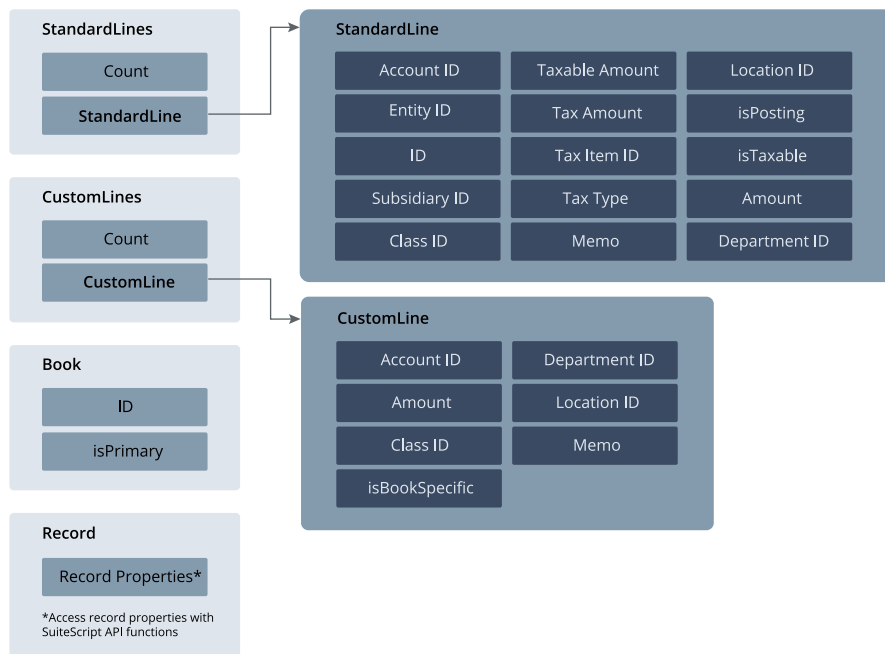
Function / Interface Object	Description
customizeGIImpact(transactionRecord, standardLines, customLines, book)	You'll need to create an implementation of this function in the plug-in script file. The script can contain other helper functions. NetSuite runs this function when you save a transaction of the type and subsidiary you set up for the plug-in implementation. NetSuite runs this function one time for the primary accounting book. If you use Multi-Book Accounting, it also runs for each secondary accounting book you set up for the plug-in implementation.
StandardLines	Contains an array of all standard lines with GL impact in a transaction as StandardLine objects.

Function / Interface Object	Description
	Use the methods available to this object to access the properties of each standard line with GL impact on a transaction.
CustomLines	Contains an array of all custom lines with GL impact in a transaction as CustomLine objects. Use the methods available to this object to add and change custom lines with GL impact on a transaction.
AccountingBook	Represents the accounting book passed to a Custom GL plug-in implementation when you save a transaction. Use the methods available to the book object to determine if the book is a primary or secondary book or get the internal NetSuite ID of the accounting book.
Record	Use this to access properties of the transaction with SuiteScript API nlobjRecord functions. You can't modify the transaction. When you create a new transaction in synchronous mode, the record ID isn't available.

For more information, see the [Custom GL Lines Plug-in Process Flow](#) and [Custom GL Lines Plug-in Implementation Object Model for SuiteScript 1.0](#).

Custom GL Lines Plug-in Implementation Object Model for SuiteScript 1.0

The following diagram shows the object model for the interface input and output objects:



Rules and Guidelines

Use the following rules and guidelines when creating the plug-in implementation script file:

- The plug-in script file can have any name, provided it has an implementation of the interface function.
- If you want to create utility files with helper functions for the main implementation file, you can include them when you create the plug-in implementation for the Custom GL Lines plug-in in NetSuite. For more information about utility files, see [Utility Files for a Custom GL Lines Plug-In Implementation](#).
- You can use a custom record type to store the references to the required NetSuite standard records. See [Using a Custom Record to Reference Internal NetSuite IDs Example](#).
- You can use SuiteScript API functions in a plug-in implementation, for example, `nlapiloadRecord(type, id, initializeValues)` and `nlapiSearchRecord(type, id, filters, columns)`. For more information, see  [SuiteScript 1.0 API Reference](#) and [SuiteScript 2.x API Reference](#). However, using these APIs may slow down your plug-in, so try to use them only when needed to improve performance of the plug-in implementation. For better performance searching for records is better than loading NetSuite records. These functions also have governance limits. The plug-in script file lets you use up to 1000 usage units with SuiteScript. For more information, see SuiteScript 1.0 API Governance in  [SuiteScript 1.0](#) and [SuiteScript 2.x API Governance](#).



Important: For additional guidelines and best practices, see [Custom GL Lines Plug-in Guidelines and Best Practices](#).

Plug-in Implementation Script File Creation for SuiteScript 2.0

You can use the SuiteCloud IDE, or another JavaScript IDE or text editor, to create a JavaScript file with your Custom GL Lines plug-in implementation's business logic.

The following table describes the functions and objects available in a Custom GL Lines plug-in implementation script file:

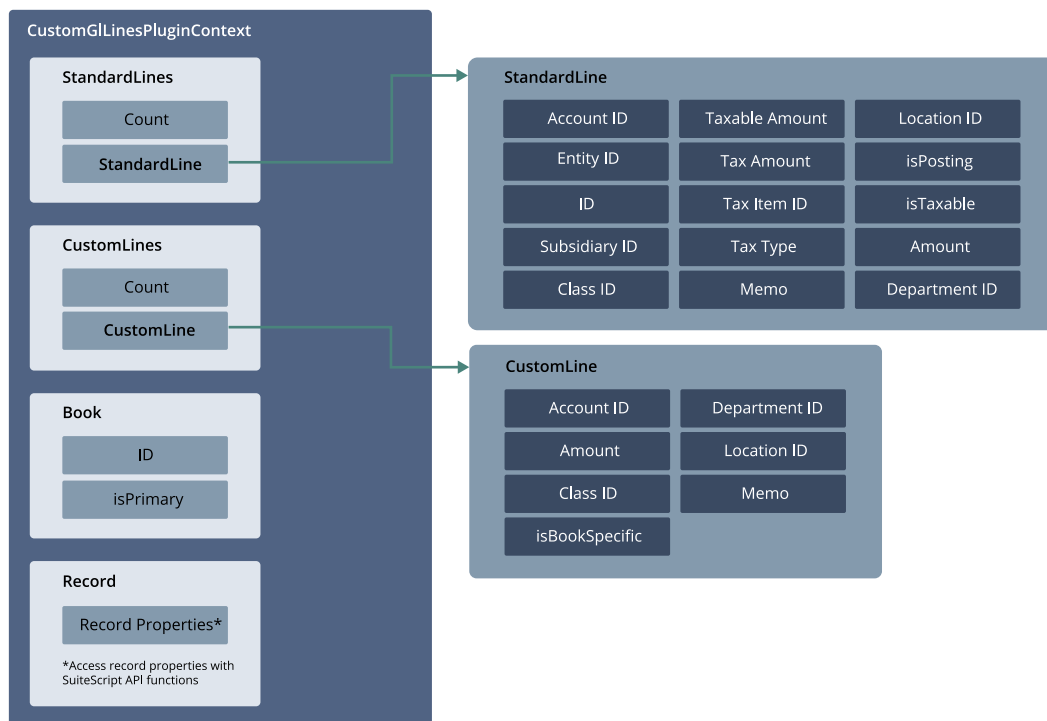
Function / Interface Object	Description
customizeGLImpact(context)	You must create an implementation of this function in the plug-in implementation script file. The script file can contain other helper functions. NetSuite runs this function when you save a transaction of the type and subsidiary you set for the plug-in implementation. NetSuite runs this function one time for the primary accounting book. If you use Multi-Book Accounting, this function also runs for each secondary accounting book that you set up for the plug-in implementation.
StandardLines	Contains an array of all standard lines with GL impact in a transaction as <code>StandardLine</code> objects. Use the methods available to this object to access properties of individual standard lines with GL impact on a transaction.
CustomLines	Contains an array of all custom lines with GL impact in a transaction as <code>CustomLine</code> objects. Use the methods available to this object to add and modify custom lines with GL impact on a transaction.
AccountingBook	Represents the accounting book passed to a Custom GL plug-in implementation when you save a transaction. Use the methods available to

Function / Interface Object	Description
	the book object to determine if it's a primary or secondary book, or to get the internal NetSuite ID of the accounting book.
ReadOnlyTransactionRecord	Use this to access properties of the transaction with SuiteScript API <code>nlobjRecord</code> functions. You cannot modify the transaction. When you create a new transaction in synchronous mode, the record ID is not available.

For more information, see the [Custom GL Lines Plug-in Process Flow](#) and [Custom GL Lines Plug-in Implementation Object Model for SuiteScript 2.x](#).

Custom GL Lines Plug-in Implementation Object Model for SuiteScript 2.x

The following diagram shows the object model for the interface input and output objects:



Rules and Guidelines

Use the following rules and guidelines when creating the plug-in implementation script file:

- The plug-in script file can have any name, provided it has an implementation of the interface function.
- If you want to create utility files with helper functions for the main implementation file, you can include them when you create the plug-in implementation for the Custom GL Lines plug-in in NetSuite. For more information about utility files, see [Utility Files for a Custom GL Lines Plug-in Implementation](#).
- You can use a custom record type to store references to the required NetSuite standard records. See [Using a Custom Record to Reference Internal NetSuite IDs Example](#).
- You can use SuiteScript API modules in a plug-in implementation, for example, the [N/record Module](#) and [N/query Module](#). For more information, see the help topic [SuiteScript 2.x API Reference](#). However,

using these APIs may slow down your plug-in implementation, so use them only when needed to optimize the performance of the plug-in implementation. In general, searching for records yields better performance than loading NetSuite records.

These functions also have governance limits. The plug-in script file lets you use up to 1000 usage units with SuiteScript. For more information, see the help topic [SuiteScript 2.x API Governance](#).



Important: For additional guidelines and best practices, see [Custom GL Lines Plug-in Guidelines and Best Practices](#).

Adding the Plug-in Implementation

After you create the plug-in implementation script file, create a new plug-in implementation. When you create the plug-in implementation, upload the script file and other utility files as required.

To add the script files and create the plug-in implementation:

1. Go to Customization > Plug-ins > Plug-in Implementations > New.
2. In the **Script File** field, open the script file or add a new file.
3. Click **Create Plug-in Implementation**.

Select 1.0 Plug-in Type	
Type	Description
Custom GL Lines Plug-in	Allow for GL impact customization
Email Capture	Process emails sent to specified email capture address
OCR Plugin	OCR plugin
Docusign	
MX UUID	

4. On the Select Plug-in Type page, click **Custom GL Lines Plug-in**.
5. On the New Plug-in Implementation page, enter the following information:

Option	Description
Name	User-friendly name for the implementation. The plug-in implementation appears in the following locations: - Manage Plug-ins page. Page used by administrators to enable/disable the plug-in implementation in their account. - Bundle Builder. Select this name in the Bundle Builder to distribute the plug-in implementation to other accounts. - GL Impact page. For each custom lines added by the plug-in implementation, the Custom Script column on the GL Impact page lists the plug-in implementation name.
ID	Internal ID for the implementation, used in scripting. If you don't provide one, NetSuite will provide an ID when you click Save .
Status	Current status for the implementation. Choose Testing to make it accessible only to the owner of the implementation. Choose Released to make it available to everyone in the account.
Log Level	Logging level for the script: select Debug, Audit, Error, or Emergency. These messages appear on the Execution Log subtab for the plug-in implementation.
Execute As Role	Role that the script runs as. The Execute As Role field controls the permissions and restrictions for the script. If you select Current Role , the script runs using the permissions of the user who is currently logged in.

Option	Description
	 Note: You can create a custom role for testing, provided it has the SuiteScript permission. You can bundle the custom role with the plug-in implementation if you want to distribute it. See Test the Plug-in Implementation and Bundle the Plug-in Implementation .
Description	Optional description of the implementation that appears on the Plug-In Implementations page.
Owner	User account that owns the implementation. By default it's the name of the logged-in user.
Inactive	Indicates the plug-in implementation does not run in the account. Inactivate a plug-in implementation to temporarily disable it for testing.

- On the Scripts subtab, from the **Implementation** list, select the script file with the plug-in implementation.
- On the Scripts subtab, from the **Library Script File** list, select any utility script files needed by the plug-in script file.
- On the Unhandled Errors subtab, select who is notified if script errors occur. By default the **Notify Script Owner** box is checked.
To add more than one email address in the **Notify Emails** box, separate them with a semicolon.
- Click **Save**. You can see all implementations by going to Customization > Plug-ins > Plug-in Implementations.

Before you can test the plug-in implementation, configure and enable it. See [Configuring the Custom GL Lines Plug-in Implementation](#) and [Activate the Plug-in](#).

 **Note:** You can use SuiteCloud Development Framework (SDF) to manage saved custom GL lines plug-in implementations as part of file-based customization projects. For information about SDF, see .

You can use the Copy to Account feature to copy one custom GL lines plug-in implementation to another account. When you edit a custom GL lines plug-in implementation, you'll see a **Copy to Account** option in the upper right corner. For more information, see .

Test the Plug-in Implementation

To test a Custom GL Lines plug-in implementation, perform the following tasks:

- Create a custom role for the plug-in implementation. The **Execute As Role** field determines the role which the plug-in implementation script runs as, and it needs the SuiteScript permission.
For more information about the **Execute As Role** field, see [Adding the Plug-in Implementation](#). For more information about custom roles in NetSuite, see the help topic [Customizing or Creating NetSuite Roles](#).
- Configure the transaction types, subsidiaries, and accounting books for the plug-in implementation, based on your accounting features. See [Configuring the Custom GL Lines Plug-in Implementation](#).
- Enable the plug-in implementation at Customization > Plug-ins > Manage Plug-ins. See [Activate the Plug-in](#).
- Create transactions that impact the general ledger and test the script and plug-in behavior.
- Examine the GL impact of the transactions and make sure the plug-in implementation works as expected.



Important: Plug-in implementations modify or supplement NetSuite business logic, changing the manner in which standard business processes run. Make sure that the GL impacts on the transaction types, subsidiaries, and accounting books match the customization in the plug-in implementation script file and configuration.

You should test your plug-in implementation on every NetSuite version and release your customers use.

If you use the Custom GL Lines Plug-in with Custom Segments, the plug-in may fail for any of the following reasons:

- Custom segment does not exist.
- Custom segment is inactive.
- The role set up to run the plug-in doesn't have permission to set values on the custom segment.
- The custom segment doesn't impact GL.
- The Custom Segment feature isn't enabled.

Bundle the Plug-in Implementation

After you've developed the Custom GL Lines plug-in implementation, you can distribute it to a production account. SuiteBundler lets NetSuite users package groups of objects to distribute to other accounts. These packages are called bundles, or SuiteApps. To distribute the Custom GL Lines plug-in implementation script file, utility files, custom records and roles, and the plug-in implementation object, create a bundle with SuiteBundler. After you create the bundle, administrators can install it in production accounts.

The following table lists the objects you must include in the bundle and their location on the **Select Objects** page in the Bundle Builder:

Object	Location On Select Objects Page
Plug-in implementation script file	File Cabinet > Files
Utility files	
Custom records	Custom Lists/Records > Records
Custom roles	Roles > Custom Roles
Plug-in implementation	Plug-ins > Custom GL Lines Plug-in

Custom GL Lines Plug-in Implementation Administration

After a developer creates and bundles a Custom GL Lines plug-in implementation, you can install the bundle and set up the plug-in implementation.

To install and set up a Custom GL Lines plug-in implementation, complete the following steps:

- [Enabling Features for a Custom GL Lines Plug-in Implementation](#)
- [Install a Shipping Partner Plug-in Bundle](#)
- [Configuring the Custom GL Lines Plug-in Implementation](#)

■ Activate the Plug-in

You can monitor implementations using system notes. For information, see [Custom GL Lines Plug-in Implementation System Notes](#).



Important: The system always uses the current version of the Custom GL Lines Plug-in. An older version of the plug-in may have been used to create a transaction, which may give you a different result than the current version. If you try to update a transaction in a closed period and the current plug-in generates a different result, you can't save the updates.

You can either open the period or branch your plug-in implementation by the date of the plug-in version.

Enabling Features for a Custom GL Lines Plug-in Implementation

Before you install a Custom GL Lines Plug-in bundle, make sure that the Custom GL Lines Plug-in and Server SuiteScript features are enabled in your account.

To enable features for the Custom GL Lines Plug-in:

1. Choose Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, enable the following features:

Section	Feature Name
SuiteScript	Server SuiteScript
SuiteGL	Custom GL Lines

3. If needed, check the box and agree to the Terms of Service.
4. Click **Save**.

Installing a Custom GL Lines Plug-in Bundle

A developer can create a Custom GL Lines plug-in implementation and bundle it to distribute to other NetSuite accounts. Then, an administrator can install the bundle in the target NetSuite account.

To install a Custom GL Lines plug-in implementation bundle:

1. Go to Customization > SuiteBundler > Search & Install Bundles.
2. On the Install Bundle page, choose Production Account from the **Location** list.
3. Search for the plug-in bundle.
4. Click **Install** for the bundle.



Important: To avoid duplicate objects during installation, select **Replace Existing Object** if you're prompted.

After you start installing a bundle, you can continue working in NetSuite as the bundle installs.

To check the installation progress, go to the list of installed bundles at Customization > SuiteBundler > Search & Install Bundles > List. If installation isn't done yet, the Status column

displays the percentage complete. Click Refresh to update the status. When installation is complete, you'll see a green check mark in the Status column.

Note: If you don't see an Install button, the bundle may not have been shared with your account. To get access, contact the developer or NetSuite Customer Support.

Important: If you uninstall the bundle with the plug-in implementation and it created any custom lines, those custom lines remain in your account after the plug-in implementation is uninstalled. For more information, see [Deleting a Custom GL Lines Plug-in Implementation](#).

Configuring the Custom GL Lines Plug-in Implementation

Configure the transaction types, subsidiaries, and accounting books for a Custom GL Lines plug-in implementation from the Plug-In Implementation page. When you save a transaction, any configured transaction type with an entity associated with a configured subsidiary triggers the plug-in implementation. If you use Multi-Book Accounting, NetSuite runs the plug-in implementation logic on each accounting book you set up for it.

For more information about how the transaction types, subsidiaries, and accounting books affect the running of a plug-in implementation, see [Custom GL Lines Plug-in Process Flow](#).

Note: You need the SuiteScript permission to configure a Custom GL Lines plug-in implementation.

To configure a Custom GL Lines plug-in implementation:

1. Go to Customization > Plug-ins > Plug-in Implementations.
2. Click **Edit** next to the Custom GL Lines plug-in implementation that you want to configure.
3. Click **Configure**. The Custom GL Lines: Configuration page appears.

4. By default, plug-ins run synchronously. If you have multiple plug-ins on a transaction type, saving may take a long time. If a transaction type takes a long time to save, you can set one or more plug-in implementations to run asynchronously.

Every plug-in runs independently and has access only to its own lines and the standard lines. If one plug-in fails, the others can still run. No priority is assigned to individual plug-ins, meaning that they run in any order.

5. By default, custom GL lines plug-ins are not run in closed periods and the original GL impact of custom GL lines stays the same. To run the custom GL lines plug-in on transactions in closed periods, check the **Run in Closed Periods** box. If you check the box and then edit a transaction, you'll see an error message indicating that the GL impact can't be changed in a closed period.

 **Note:** The Run in Closed Periods setting affects only transactions that have a posting period.

6. The **Effective Date** and **End Date** fields help when you're updating or adding a script file for an implementation. In the **Effective Date** field, enter a start date. The plug-in runs on transactions dated on or after this date. This field is blank by default and isn't required.
7. To edit or inactivate the plug-in and preserve the existing GL impact, enter an end date for the older plug-in in the **End Date** field. The plug-in won't run on transactions after this date. This field is blank by default and isn't required.

If you're loading a new script for a Custom GL Plug-in implementation, enter an end date for the older version. Then, add the new script and create a new plug-in implementation with an effective date that's the day after the end date of the old plug-in implementation.

If you update the script used by the existing plug-in implementation file, all plug-in implementations that use that file are affected. It's best to add a new script file and create a new plug-in implementation with a different effective date.

 **Note:** If one plug-in implementation's end date is the same as another plug-in's effective date, both plug-ins run on that day. To avoid overlap, set the effective date of the new plug-in to one day after the old plug-in's end date.

8. Select the transaction types on which the plug-in implementation executes.
To select multiple or all transaction types, hold the Ctrl key and select the transaction types.
Check the **ALL** box to select all of the transactions types for this configuration.
9. In the **Subsidiaries** list, select one or more subsidiaries, or check **ALL**.
10. In the **Accounting Books** list, select one or more accounting books, or check **ALL**.
11. Click **Save**.

When you configure additional plug-ins that overlap with an existing one, you'll see a message about running the plug-in asynchronously. You can save the configuration, but if saving transactions takes a long time, try making one or more plug-in implementations asynchronous to speed the save process.

Enabling the Custom GL Lines Plug-in Implementation

After you have configured a Custom GL Lines plug-in implementation, you can enable it in your account.

 **Note:** You can have more than one Custom GL Lines plug-in implementations enabled in your account. If you enable a plug-in implementation that conflicts with another implementation, NetSuite displays a warning message that having several implementations for one transaction can slow down save times. You can save the implementation, but if saving transactions takes too long, try setting one or more plug-in implementations to run asynchronously.

To enable the Custom GL Lines plug-in implementation:

1. Go to Customization > Plug-ins > Manage Plug-ins.

2. Under Custom GL Lines Plug-in, check the box next to the name of the plug-in implementation.
3. Click **Save**.

Note: The plug-in implementation runs when users save transactions configured for it, based on the plug-in logic. Users can see the custom lines added or changed by the plug-in implementation when they click Actions > GL Impact on a saved transaction.

Custom GL Lines Plug-in Implementation System Notes

Note: This topic applies to System Notes only. For information about System Notes v2, see the help topic [System Notes v2 Overview](#).

System notes are logged on custom GL lines plug-in implementations.

A System Notes subtab is available on the plug-in implementation configuration page for Custom GL Lines Plug-ins. The system notes show any changes made to the plug-in implementation configuration settings.

Custom GL Lines: Configuration

Save

Cancel

Name

Cash Basis on Invoice Payment

ID

customscript_cash_basis_impl

Status

Testing

Log Level

Debug

☐ Asynchronous

☐ Run in Closed Periods

Effective Date

End Date

Execution Rules

System Notes

View

Field

Default

- All -

Customize View

Date ↓	Set by	Context	Type	Field	Old Value	New Value
5/12/2025 11-29	C Wolfe	UI	Set	Asynchronous		F
5/12/2025 11-29	C Wolfe	UI	Set	All Transaction Types		F
5/12/2025 11-29	C Wolfe	UI	Set	Run In Closed Periods		F

For more information about system notes, see the help topic [System Notes Overview](#).

Custom GL Lines Plug-In Interface Definition

Interface Function

The Custom GL Lines plug-in interface includes the following functions for SuiteScript 1.0:

Function	Description
customizeGImpact(transactionRecord, standardLines, customLines, book)	Contains the business logic to modify the general ledger impact for a transaction with the Custom GL Lines plug-in. For more information about how NetSuite uses this function, see Custom GL Lines Plug-in Process Flow.



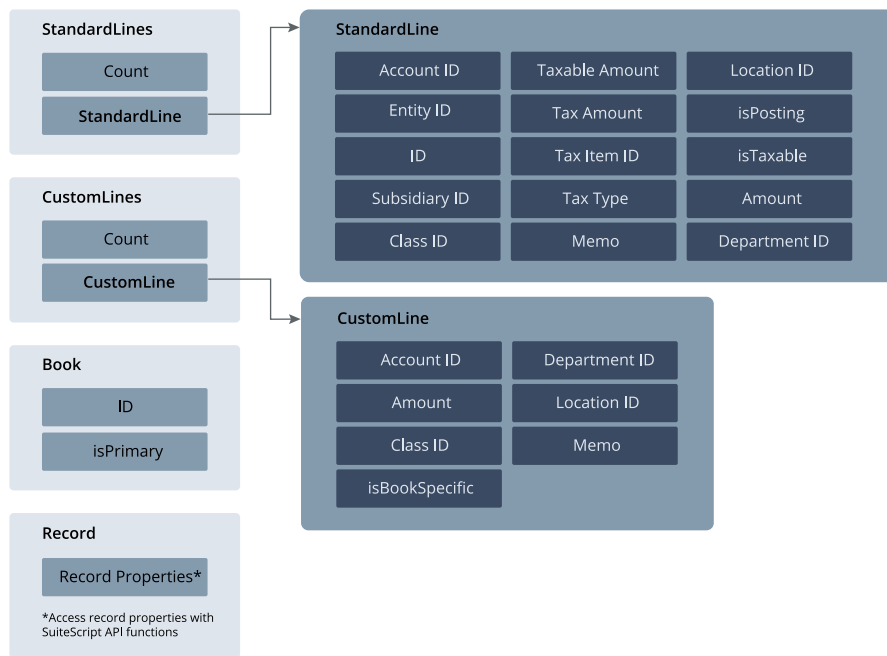
Important: You cannot change this function signature in a Custom GL Lines plug-in implementation.

The Custom GL Lines plug-in interface includes the following function for SuiteScript 2.x:

Function	Description
customizeGImpact(context)	Contains the business logic to modify the general ledger impact for a transaction with the Custom GL Lines plug-in. For more information about how NetSuite uses this function, see Custom GL Lines Plug-in Process Flow.

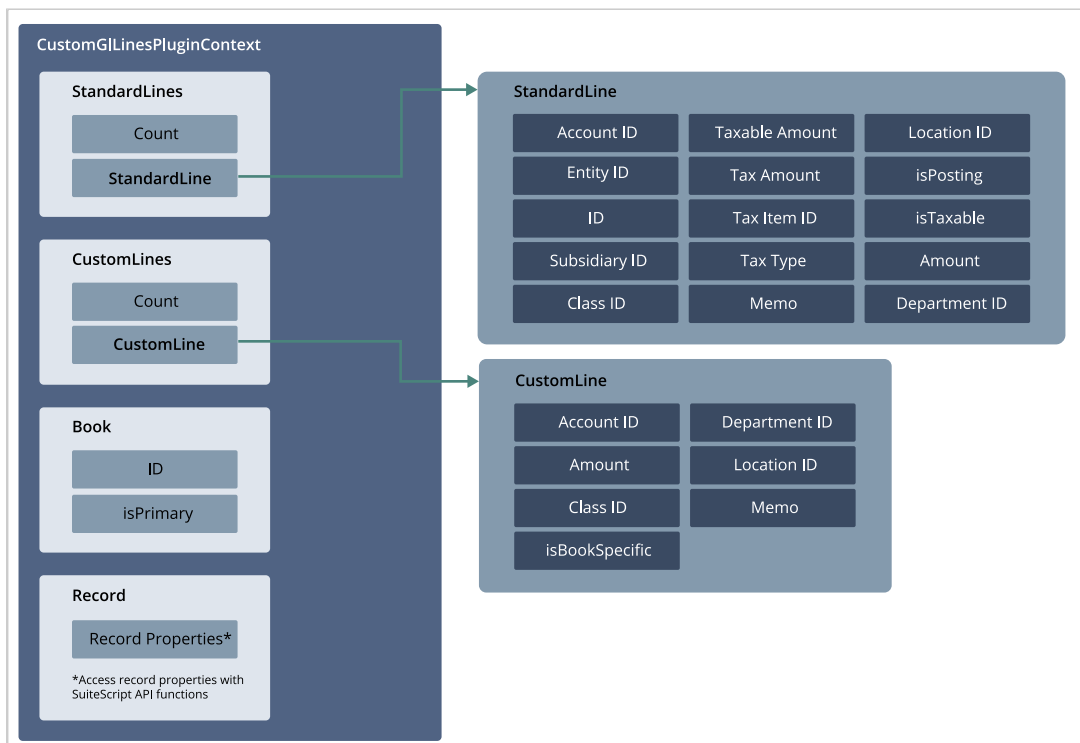
Custom GL Lines Plug-In Object Model for SuiteScript 1.0

The following figure shows the object model for the interface input objects in SuiteScript 1.0:



Custom GL Lines Plug-In Object Model for SuiteScript 2.x

The following figure shows the object model for the interface input objects in SuiteScript 2.x:



customizeGImpact(transactionRecord, standardLines, customLines, book)

Function Declaration	void customizeGImpact(Record transactionRecord, StandardLines standardLines, CustomLines customLines, AccountingBook book)
Type	Interface function
Description	Contains the business logic to modify the general impact for a transaction with the Custom GL Lines Plug-in. Use the objects passed to this function by NetSuite to implement the following business logic in a Custom GL Lines plug-in implementation: ■ Get information about general ledger impact. Use the StandardLines object to get information about each line of general ledger impact for a transaction. ■ Create custom lines. Use the CustomLines object to create lines with general ledger impact in primary and secondary accounts for a transaction. ■ Modify how GL impact in the primary book affects the secondary books. You can create different business logic for primary and secondary accounting books. See AccountingBook.  Note: You can also use SuiteScript API functions to access transaction record properties in a plug-in implementation. See Record.
Returns	void
Parameters	■ StandardLines ■ CustomLines ■ AccountingBook ■ Record

StandardLines

Type	Interface input object
Description	Contains an array of all standard lines with GL impact in a transaction as StandardLine objects. Standard lines are the general ledger impacts that appear on the GL Impact report for a transaction. Use this object to access individual standard lines for an accounting book in a transaction. For more information, see Custom GL Lines Plug-in Process Flow.
Methods	■ getCount() ■ getLine(index)
Child Object(s)	StandardLine

getCount()

Function Declaration	int getCount()
Type	Object method

Description	Returns the number of standard lines with GL impact for a specific accounting book in a transaction. Use this method in conjunction with getLine(index) to read individual standard lines. getCount() counts both visible and hidden lines on the GL Impact page. For an example of how to return only the number of visible lines, see Custom GL Lines Plug-in Guidelines and Best Practices .
Returns	int
Input Parameters	None.
Parent object	StandardLines

Example

```

1  function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          ...
8      }
9  }

```

getLine(index)

Function Declaration	StandardLine getLine(int index)
Type	Object method
Description	Returns a StandardLine object that represents a standard line with GL impact. StandardLine objects are stored in the StandardLines object starting at index 0.
Returns	StandardLine
Input Parameters	index {int} — Integer value for the StandardLine object at the specified position in the StandardLines object.
Parent object	StandardLines

Example

```

1  function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          ...
8      }
9  }

```

StandardLine

Type	Object
Description	Contains all properties for a single standard line on the GL impact on a transaction. Use the methods available to the StandardLine object to get the values for the standard line and define

plug-in implementation functionality based on the values. The StandardLines object has a StandardLine object for each standard GL impact line.

Methods

- [getEntityId\(\)](#)
- [getId\(\)](#)
- [getSubsidiaryId\(\)](#)
- [getTaxableAmount\(\)](#)
- [getTaxAmount\(\)](#)
- [getTaxItemId\(\)](#)
- [getTaxType\(\)](#)
- [isPosting\(\)](#)
- [isTaxable\(\)](#)



Note: The StandardLine and CustomLine objects share some common methods. For more information, see [Common StandardLine and CustomLine Object Methods](#).

Parent Object(s) [StandardLines](#)

Child Object(s) n/a

getEntityId()

Function Declaration `int getEntityId()`

Type Object method

Description Returns the internal NetSuite ID of the entity for a standard line. The entity can be any customer, employee, partner, or vendor, depending on the type of transaction.

You can use this ID to define plug-in implementation functionality based on the entity ID, load the record for the entity with `nlapiloadRecord(type, id, initializeValues)`, or search for the record with `nlapisearchRecord(type, id, filters, columns)`. You can look up the entity type based on the record type using `getRecordType()`. See the help topic [SuiteScript 1.0 Documentation](#).

See also [Record](#).



Important: Limit using these APIs to improve the performance of your plug-in implementation. Searching for a record is faster than loading it.

To implement functionality based on the entity ID, you can use helper functions to avoid adding the internal NetSuite ID in the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns `int`

Input Parameters None.

Parent object [StandardLine](#)

Example 1: Loading the Record

```
1  function customizeGIImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
```

```

6      var currLine = standardLines.getLine(i);
7      var recType = transactionRecord.getRecordType();
8      if (recType == 'salesorder')
9      {
10         var custRecord = nlapiLoadRecord('customer', currLine.getEntityId());
11         // get record properties here
12         ...
13     }
14     ...
15 }
16 }

```

Example 2: Searching for the Record

```

1  // utility function
2  function searchNeededPropertiesOfCustomer(customerId)
3  {
4      var searchFilters = new Array();
5      searchFilters[0] = new nlobjSearchFilter('internalid', null, 'anyof', customerId);
6      var savedSearchResults = nlapiSearchRecord('customer', 'customsearch_script_id', searchFilters);
7
8      if (savedSearchResults != null && savedSearchResults.length > 0)
9      {
10         return savedSearchResults[0];
11     }
12     return null;
13 }
14
15 function customizeGllImpact(transactionRecord, standardLines, customLines, book)
16 {
17     ...
18     for (var i = 0; i < standardLines.getCount(); i++)
19     {
20         var currLine = standardLines.getLine(i);
21         var recType = transactionRecord.getRecordType();
22         if (recType == 'salesorder')
23         {
24             // nlobjSearchResult
25             var custMap = searchNeededPropertiesOfCustomer(currLine.getEntityId());
26             // get search columns here
27             ...
28         }
29         ...
30     }
31 }

```

getId()

Function Declaration int getId()

Type Object method

Description Returns the internal NetSuite database ID for a standard GL impact line. For most transaction types, the summary line for the GL impact has an ID of 0, but some transactions, like journals, don't have a summary line.

You can use the summary information to define plug-in implementation functionality based on the summary line values.



Note: The summary line for the GL impact on a transaction does not include tax information.

Returns int

Input Parameters None.

Parent object [StandardLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          if (currLine.getId() == 0)
8          {
9              // it's a summary line
10             }
11             ...
12         }
13     }

```

getSubsidiaryId()

Function Declaration int getSubsidiaryId()

Type Object method

Description Returns the internal NetSuite ID of the subsidiary for the entity associated with a standard GL impact line. You can view all subsidiaries and their internal IDs in NetSuite at Setup > Company > Subsidiaries.

You can use this ID to define plug-in implementation functionality based on the subsidiary ID, load the record for the subsidiary with `nlapiLoadRecord(type, id, initializeValues)`, or search for the record with `nlapiSearchRecord(type, id, filters, columns)`. See the help topic [SuiteScript 1.0 Documentation](#).

The type parameter for a subsidiary is subsidiary.



Important: Limit using these APIs to improve the performance of your plug-in implementation. Searching for a record is faster than loading it.

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns int

Input Parameters None.

Parent object [StandardLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          var subId = currLine.getSubsidiaryId();
8          ...
9          var subRecord = nlapiLoadRecord('subsidiary', subId, '');
10         ...
11     }
12     ...
13 }
14

```


getTaxableAmount()

Function Declaration string getTaxableAmount()

Type Object method

Description Returns a string showing the amount of a standard GL line that was taxed. Returns 0 if the line wasn't taxed. Use this method with [getTaxAmount\(\)](#), [getTaxItemId\(\)](#), and [getTaxType\(\)](#) to get the tax information for a standard GL impact line.

You can use helper functions in a utility file so you don't have to add internal NetSuite IDs in the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns string

Input Parameters None.

Parent object [StandardLine](#)

Example

```

1  function customizeGIImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          // check tax
8          if (currLine.getTaxItemId() != -7) // taxable transaction
9          {
10             var taxType = currLine.getTaxType();
11             if (taxType == "US_ST")
12             {
13                 // process state taxes
14                 var taxAmt = currLine.getTaxAmount();
15                 var taxableAmt = currLine.getTaxableAmount();
16                 ...
17             }
18         }
19         ...
20     }
21 }
```

getTaxAmount()

Function Declaration string getTaxAmount()

Type Object method

Description Returns a string showing the amount of tax charged on a standard GL line. This method only returns a value other than 0 if the standard line is a credit to a tax account. For example, if a standard line on a taxable transaction has a credit value of 10.00 for an account of type Other Current Liability, this method returns -10.

Use this method with [getTaxableAmount\(\)](#), [getTaxItemId\(\)](#), and [getTaxType\(\)](#) to get the tax information for a standard GL impact line.

Returns string

Input Parameters None.

Parent object [StandardLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          // check tax
8          if (currLine.getTaxItemId() != -7) // taxable transaction
9          {
10             var taxType = currLine.getTaxType();
11             if (taxType == "US_ST")
12             {
13                 // process state taxes
14                 var taxAmt = currLine.getTaxAmount();
15                 var taxableAmt = currLine.getTaxableAmount();
16                 ...
17             }
18         }
19         ...
20     }
21 }

```

getTaxItemId()

Function Declaration `int getTaxItemId()`

Type Object method

Description Returns the internal NetSuite ID of the tax code for a standard GL line. Returns null if the line wasn't taxed or if the line doesn't credit a tax account with a tax liability.

You can use this ID to define plug-in implementation functionality based on the tax code. For example, a tax code of -7 means the line wasn't taxed. You can view all tax codes in NetSuite at Setup > Accounting > Tax Codes.

You can use helper functions in a utility file so you don't have to add internal NetSuite IDs in the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Use this method with [getTaxableAmount\(\)](#), [getTaxAmount\(\)](#), and [getTaxType\(\)](#) to get the tax information for a standard GL impact line.

Returns `int`

Input Parameters None.

Parent object [StandardLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          // check tax
8          if (currLine.getTaxItemId() != -7) // taxable transaction
9          {
10             var taxType = currLine.getTaxType();
11             if (taxType == "US_ST")
12             {
13                 // process state taxes
14                 var taxAmt = currLine.getTaxAmount();
15                 var taxableAmt = currLine.getTaxableAmount();

```

```

16         ...
17     }
18 }
19 ...
20 }
21 }

```

getTaxType()

Function Declaration	string getTaxType()
Type	Object method
Description	Returns the tax type for a standard GL line that was taxed. Returns null if the standard line wasn't taxed. You can use this ID to define plug-in implementation functionality based on the tax type. You can view all tax types in NetSuite at Setup > Accounting > Tax Types. Use this method with getTaxableAmount(), getTaxAmount(), and getTaxItemId() to get the tax information for a standard GL impact line.
Returns	string
Input Parameters	None.
Parent object	StandardLine

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var currLine = standardLines.getLine(i);
7          // check tax
8          if (currLine.getTaxItemId() != -7) // taxable transaction
9          {
10             var taxType = currLine.getTaxType();
11             if (taxType == "US_ST")
12             {
13                 // process state taxes
14                 var taxAmt = currLine.getTaxAmount();
15                 var taxableAmt = currLine.getTaxableAmount();
16                 ...
17             }
18         }
19         ...
20     }
21 }

```

isPosting()

Function Declaration	boolean isPosting()
Type	Object method
Description	Returns true if the transaction is a posting transaction and the associated standard GL impact line posts to the general ledger. Returns false if it's a non-posting transaction. NetSuite transactions can be posting, non-posting, or posting with approval. For example, journal entries don't post without approval. For more information, see the help topic General Ledger Impact of Transactions.

Returns boolean

Input Parameters None.

Parent object [StandardLine](#)

Example

```

1  function customizeGIImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      //find all posting lines
5      for (var i = 0; i < standardLines.getCount(); i++)
6      {
7          var currLine = standardLines.getLine(i);
8          if (currLine.isPosting())
9          {
10             // process standard line here
11         }
12     }
13     ...
14 }
```

isTaxable()

Function Declaration boolean isTaxable()

Type Object method

Description Returns true if a standard GL impact line is a credit to a tax account. For example, if a standard line on a taxable transaction has a credit value of 10.00 for an account of type Other Current Liability, this method returns true.

Returns boolean

Input Parameters None.

Parent object [StandardLine](#)

Example

```

1  function customizeGIImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      //find all taxable lines
5      for (var i = 0; i < standardLines.getCount(); i++)
6      {
7          var currLine = standardLines.getLine(i);
8          if (currLine.isTaxable())
9          {
10             // process tax here
11         }
12     }
13     ...
14 }
```

CustomLines

Type Interface input object

Description	Contains an array of all custom lines with GL impact in a transaction as CustomLine objects. Use this object to add and modify custom lines with GL impact on a transaction. Create a new CustomLine object with addNewLine() . For more information, see Custom GL Lines Plug-in Process Flow .
Methods	■ addNewLine() ■ getCount() ■ getLine(index)
Child Object(s)	CustomLine

addNewLine()

Function Declaration	CustomLine addNewLine()
Type	Object method
Description	Adds a CustomLine object to the parent CustomLines object in a Custom GL Lines plug-in implementation and returns the new object. Use this method to add a custom line with GL impact to a transaction. After you create the custom line, use the methods available to the CustomLine object to set the properties of the custom line, including the general ledger account ID and the amount of the custom line.  Note: If you use the Multi-Book Accounting feature and want this line to be copied to the secondary accounting books, use setBookSpecific(bookSpecific).
Returns	CustomLine
Input Parameters	None.
Parent object	CustomLines

Example

<pre> 1 function customizeGllImpact(transactionRecord, standardLines, customLines, book) 2 { 3 ... 4 var newLine = customLines.addNewLine(); 5 newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount()); 6 newLine.setAccountId(standardLines.getLine(0).getAccountId()); 7 newLine.setMemo("Payment catches both revenue and cash."); 8 ... 9 var newLine = customLines.addNewLine(); 10 newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount()); 11 newLine.setAccountId(standardLines.getLine(1).getAccountId()); 12 newLine.setMemo("Payment catches both revenue and cash."); 13 ... 14 } </pre>	
---	--

getCount()

Function Declaration	int getCount()
Type	Object method

Description	Returns the number of custom lines with GL impact for a specific accounting book in a transaction. These lines are added with addNewLine() . Use this method in conjunction with getLine(index) to get individual custom lines after you add them in the plug-in implementation.
Returns	int
Input Parameters	None.
Parent object	CustomLines

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < customLines.getCount(); i++)
5      {
6          // get the line
7          var currLine = customLines.getLine(i);
8          ...
9      }
10 }

```

getLine(index)

Function Declaration	CustomLine getLine(int index)
Type	Object method
Description	Returns a CustomLine object that represents a custom line with GL impact. CustomLine objects are stored in the CustomLines object starting at index 0.
Returns	CustomLine
Input Parameters	index {int} — Integer value for the CustomLine object at the specified position in the CustomLines object.
Parent object	CustomLines

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < customLines.getCount(); i++)
5      {
6          // get the line
7          var currLine = customLines.getLine(i);
8          ...
9      }
10 }

```

CustomLine

Type	Object
Description	Contains all properties for a single custom line for the GL impact on a transaction. Use the methods available to the CustomLine object to set values for the custom line and define plug-

in implementation functionality based on those values. The [CustomLines](#) object contains a reference to each custom GL impact line.

Create a new CustomLine object with [addNewLine\(\)](#).

Methods

- [isBookSpecific\(\)](#)
- [setBookSpecific\(bookSpecific\)](#)
- [setAccountId\(accountId\)](#)
- [setClassId\(classId\)](#)
- [setCreditAmount\(credit\)](#)
- [setDebitAmount\(debit\)](#)
- [setDepartmentId\(departmentId\)](#)
- [setEntityId\(entityId\)](#)
- [setLocationId\(locationId\)](#)
- [setMemo\(memo\)](#)
- [setSegmentValueId\(segmentId, segmentValueId\)](#)



Note: The StandardLine and CustomLine objects share some common methods. For more information, see [Common StandardLine and CustomLine Object Methods](#).

Parent Object(s) [CustomLines](#)

Child Object(s) n/a

isBookSpecific()

Function Declaration `boolean isBookSpecific()`

Type Object method

Description Returns true if the custom line is specific to the primary book and shouldn't be copied to the secondary accounting books. Returns false if the custom line isn't specific to the primary accounting book.



Note: This method applies only if you use Multi-Book Accounting and the method is called for the primary accounting book. For more information about how the plug-in implementation processes book-specific lines, see [Custom GL Lines Plug-in Process Flow](#).

Returns boolean

Input Parameters None.

Parent object [CustomLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      if(book.isPrimary())
5      {
6          // set all custom lines to apply to all books
7          for (var i = 0; i < customLines.getCount(); i++)
8          {

```

```

9         var currLine = customLines.getLine(i);
10        if (currLine.isBookSpecific())
11        {
12            currLine.setBookSpecific(false);
13            ...
14        }
15    }
16 }
17 ...
18 }

```

setBookSpecific(bookSpecific)

Function Declaration void setBookSpecific(boolean bookSpecific)

Type Object method

Description Sets a custom GL impact line to affect only the primary book in a Custom GL plug-in implementation. If you use this method and set the value to false, the plug-in implementation copies the custom line to secondary books and applies any mapping set up for the Multi-Book Accounting feature.

If you don't use this method or set the value to true, the custom GL impact line applies only to the primary accounting book and isn't copied to secondary accounting books.

Returns void

Input Parameters bookSpecific {boolean} — Use true to make the custom line specific to the primary accounting book. Use false to direct NetSuite to copy the custom line to secondary accounting books.

Parent object [CustomLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      if (book.isPrimary())
5      {
6          // set all custom lines to be specific to the primary book
7          for (var i = 0; i < customLines.getCount(); i++)
8          {
9              var currLine = customLines.getLine(i);
10             if (!currLine.isBookSpecific())
11             {
12                 currLine.setBookSpecific(true);
13                 ...
14             }
15         }
16     }
17     ...
18 }

```

setAccountId(accountId)

Function Declaration void setAccountId(int accountId)

Type Object method

Description Sets the account ID property for a [CustomLine](#) object in a primary or secondary book. This is the internal NetSuite ID for a general ledger account. You must use this method to set the account that's debited or credited for a custom line.

You can view the internal NetSuite ID for all accounts on the Chart of Accounts page at Setup > Accounting > Chart of Accounts.

You can use helper functions in a utility file to customize the internal NetSuite account IDs in the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns	void
Input Parameters	accountId {int} — Internal NetSuite ID for an account.
Parent object	CustomLine

Example

```

1  function customizeGIImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());
6      newLine.setAccountId(standardLines.getLine(0).getAccountId());
7      newLine.setMemo("Payment catches both revenue and cash.");
8      ...
9      var newLine = customLines.addNewLine();
10     newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
11     newLine.setAccountId(standardLines.getLine(1).getAccountId());
12     newLine.setMemo("Payment catches both revenue and cash.");
13     ...
14 }

```

setClassId(classId)

Function Declaration	void setClassId(int classId)
Type	Object method
Description	Sets the class ID property for a CustomLine object in a primary or secondary book. This value is the internal NetSuite ID for a class. Classes are categories you can create to track records such as financials, transactions, and employees. You can view all classes in NetSuite at Setup > Company > Classes. See also getClassId(). You can use helper functions in a utility file so you don't have to add internal NetSuite IDs directly into the plug-in implementation logic. See Utility Files for a Custom GL Lines Plug-In Implementation.
Returns	void
Input Parameters	classId {int} — Internal NetSuite ID for a class.
Parent object	CustomLine

Example

```

1  function customizeGIImpact(transactionRecord, standardLines, customLines, book)
2  {
3      var context = nlapiGetContext();
4      var makeClassesMandatory = context.getPreference('classmandatory');
5      var allowPerLineClasses = context.getPreference('classesperline');
6      ...
7      var newLine = customLines.addNewLine();
8      ...
9      if (makeClassesMandatory || allowPerLineClasses)

```

```

10     {
11         var classId = getClassForItem(itemId);
12         newLine.setClassId(classId);
13     }
14     ...
15 }
16
17 // Utility methods
18 function getClassForItem(itemId)
19 {
20     ...
21     return classId;
22 }

```

setCreditAmount(credit)

Function Declaration void setCreditAmount(credit)

Type Object method

Description Sets the credit amount of a [CustomLine](#) object in a primary or secondary book. The value is rounded to currency precision.

To set a debit amount, use [setDebitAmount\(debit\)](#).



Note: The currency for the amount of the custom line depends on the subsidiary and accounting book to which the GL impact posts. For example, if the accounting book being processed by the plug-in implementation uses USD, the currency for the amount set by this method is USD. For more information, see [Multiple Currencies Examples](#).

Returns void

Input Parameters credit {string} — String value of a credit on a general ledger account. Requires a positive value.

Parent object [CustomLine](#)

Example

```

1  function customizeGllmpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());
6      newLine.setAccountId(standardLines.getLine(0).getAccountId());
7      newLine.setMemo("Payment catches both revenue and cash.");
8      ...
9      var newLine = customLines.addNewLine();
10     newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
11     newLine.setAccountId(standardLines.getLine(1).getAccountId());
12     newLine.setMemo("Payment catches both revenue and cash.");
13     ...
14 }

```

setDebitAmount(debit)

Function Declaration void setDebitAmount(debit)

Type Object method

Description Sets the debit amount of a [CustomLine](#) object in a primary or secondary book. The value is rounded to currency precision.

To set a credit amount, use [setCreditAmount\(credit\)](#).



Note: The currency for the amount on the custom line depends on the accounting book to which the GL impact posts. For example, if the accounting book being processed by the plug-in implementation uses USD, the currency for the amount set by this method is USD. For more information, see [Multiple Currencies Examples](#).

Returns void

Input Parameters debit {string} — String value of a debit on a general ledger account. Requires a positive value.

Parent object [CustomLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());
6      newLine.setAccountId(standardLines.getLine(0).getAccountId());
7      newLine.setMemo("Payment catches both revenue and cash.");
8      ...
9      var newLine = customLines.addNewLine();
10     newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
11     newLine.setAccountId(standardLines.getLine(1).getAccountId());
12     newLine.setMemo("Payment catches both revenue and cash.");
13     ...
14 }

```

setDepartmentId(departmentId)

Function Declaration void setDepartmentId(int departmentId)

Type Object method

Description Sets the department ID for a [CustomLine](#) object in a primary or secondary book. This value is the internal NetSuite ID for a department.

Departments are listed first on transactions, and are useful when designating transactions and employees as part of an internal team. You can view all departments defined in NetSuite at Setup > Company > Departments. See also [getDepartmentId\(\)](#).

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic. You can also access the `nlobjContext` object with `nlapigetContext()` to get the preferences for locations at Setup > Accounting > Accounting Preferences. See the help topic [SuiteScript 1.0 Documentation](#).

Returns void

Input Parameters departmentId {int} — Internal NetSuite ID for a department.

Parent object [CustomLine](#)

Example

```

1  function customizeGllImpact(transactionRecord, standardLines, customLines, book)

```

```

2  {
3    var context = nlapiGetContext();
4    var makeDepartmentsMandatory = context.getPreference('deptmandatory');
5    var allowPerLineDepartments = context.getPreference('deptsperline');
6    ...
7    var newLine = customLines.addNewLine();
8    ...
9    if (makeDepartmentsMandatory || allowPerLineDepartments)
10   {
11     var departmentId = getDepartmentForItem(itemId);
12     newLine.setDepartmentId(departmentId);
13   }
14   ...
15 }
16
17 // Utility methods
18 function getDepartmentForItem(itemId)
19 {
20   ...
21   return departmentId;
22 }

```

setEntityId(entityId)

Function Declaration	void setEntityId(int entityId)
Type	Object method
Description	Sets the entity ID property for a CustomLine object in a primary or secondary book to the internal NetSuite ID Supported entity types include: - Customers - Jobs - Leads - Prospects - Vendors - Employees - Other Names The following restrictions apply to setting of entities on custom GL lines: - Setting of a vendor is not supported on lines with an Accounts Payable account type. - Setting of a customer is not supported on lines with an Accounts Receivable account type. - Creating book-specific custom GL lines with an AP or AR account and entity combination. Only generic custom GL lines can be created with this combination. - Only entities that have the same subsidiary as a transaction can be set on lines for that transaction. - Only entities that have the same currency as a transaction can be set on lines for that transaction. - Entities must be active  Note: This method does not set the sales representative for the line, and it does not change the date of the first sale for the customer.
Returns	void
Input Parameters	entityId {int} — internal NetSuite ID for an entity

Parent object	CustomLine
---------------	------------

Example

```

1 function customizeGImpact(record, standardLines, customLines, book)
2 {
3     var line = customLines.addNewLine()
4     line.setAccountId(6)
5     line.setDebitAmount(100)
6     line.setEntityId(99)
7
8     line = customLines.addNewLine()
9     line.setAccountId(7)
10    line.setCreditAmount(100)
11 }

```

Reports that include GL information and are grouped by entity include custom GL information, if the entity is supported by the new method.

ACCOUNTING BOOK	ACCOUNT	AMOUNT (DEBIT)	AMOUNT (CREDIT)	BASE CURRENCY	POSTING	MEMO	NAME	SUBSIDIARY	DEPARTMENT	CLASS	LOCATION	CUSTOM SCRIPT
Primary Accounting Book	Petty Cash		500.00	USA	Yes		Anonymous Customer	Parent Company				
Primary Accounting Book	US only Expense	500.00		USA	Yes			Parent Company				
Primary Accounting Book	Checking		200.00	USA	Yes	Checking Account - custom line	ABC Supplies	Parent Company				Custom GL Plugin
Primary Accounting Book	VisaCard	200.00		USA	Yes	Credit Card - custom line	ABC Supplies	Parent Company				Custom GL Plugin

ACCOUNT	DATE	DOCUMENT NUMBER	ENTER	ACCOUNT	DEBIT	CREDIT	BALANCE
Checking	12/17/2015	5	ABC Supplies	Petty Cash		\$200.00	\$10,000.00
Total - Checking					\$0.00	\$200.00	\$9,800.00
Total					\$0.00	\$200.00	\$9,800.00

setLocationId(locationId)

Function void setLocationId(int locationId)

Declaration

Type Object method

Description Sets the location ID for a [CustomLine](#) object in a primary or secondary book. This value is the internal NetSuite ID for a location.

Use locations to track information about employees and transactions for multiple offices or warehouses. You can view all locations defined in NetSuite at Setup > Company > Locations. See also [getLocationId\(\)](#).

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic. You can also access the `nlobjContext` object with `nlapigetContext()` to get the preferences for locations at Setup > Accounting > Accounting Preferences. See the help topic [SuiteScript 1.0 Documentation](#).

Returns void

Input Parameters locationId {int} — Internal NetSuite ID for a location.

Parent object [CustomLine](#)

Example

```

1 function customizeGImpact(transactionRecord, standardLines, customLines, book)
2 {
3     var context = nlapigetContext();
4     var makeLocationsMandatory = context.getPreference('locmandatory');

```

```

5      var allowPerLineLocations = context.getPreference('locspeline');
6      ...
7      var newLine = customLines.addNewLine();
8      ...
9      if (makeLocationsMandatory || allowPerLineLocations)
10     {
11         var locationId = getLocationForItem(itemId);
12         newLine.setLocationId(locationId);
13     }
14     ...
15 }
16
17 // Utility methods
18 function getLocationForItem(itemId)
19 {
20     ...
21     return locationId;
22 }

```

setMemo(memo)

Function Declaration	void setMemo(string memo)
Type	Object method
Description	Sets the Memo field on a CustomLine object. See also getMemo() .
Returns	void
Input Parameters	memo {string} — String text for the Memo field.
Parent object	StandardLine , CustomLine

Example

```

1      function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2      {
3          ...
4          var newLine = customLines.addNewLine();
5          newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());
6          newLine.setAccountId(standardLines.getLine(0).getAccountId());
7          newLine.setMemo("Payment catches both revenue and cash.");
8          ...
9          var newLine = customLines.addNewLine();
10         newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
11         newLine.setAccountId(standardLines.getLine(1).getAccountId());
12         newLine.setMemo("Payment catches both revenue and cash.");
13         ...
14     }

```

setSegmentValueId(segmentId, segmentValueId)

Function Declaration	void setSegmentValueId(segmentId, segmentValueId)
Type	Object method
Description	Sets custom segment values on a CustomLine object NetSuite assigns custom segment values as follows: ■ If the custom segment has a body and lines, the value is copied from the body to all lines. The plug-in can be used to change the value of the lines. ■ If the custom segment has only a body, the value is copied from the body to all lines. The plug-in can be used to change the value.

- If the custom segment has only a line value, no values are copied.

Returns	void
Input Parameters	segmentId {string} — String value of the custom segment ID segmentValue {integer} — Integer value that the custom line should be set to
Parent object	CustomLines

Example

```

1 function customizeGllImpact(transactionRecord, standardLines, customLines, book)
2 {
3   ...
4   var newLine = customLines.addNewLine();
5   ...
6   // custom function for getting value for cost center segment
7   newLine.setSegmentValueId('csegcostcenter', getCostCenterValueForItem(itemId));
8   // using constant for region segment
9   newLine.setSegmentValueId('csegregion', 3);
10  // copying segment value from line 0 for work center segment
11  var workcenter = 'csegworkcenter'
12  newLine.setSegmentValueId(workcenter, standardLines.getLine(0).getSegmentValueId(workcenter));
13  ...
14 }
15 // Utility methods
16 function getCostCenterValueForItem(itemId)
17 {
18   ...
19   return valueId;
20 }

```

AccountingBook

Type	Interface input object
Description	Represents the accounting book passed to a Custom GL Lines plug-in implementation when you save a transaction. Use the methods available to the book object to determine if the book is a primary or secondary book or get the internal NetSuite ID of the accounting book. If you use the Multi-Book Accounting feature, the AccountingBook object represents a different accounting book each time the plug-in implementation executes. For more information, see Custom GL Lines Plug-in Process Flow.
Methods	■ getId() ■ isPrimary()
Child Object(s)	n/a

getId()

Function Declaration	int getId()
Type	Object method
Description	Returns the internal NetSuite ID for the accounting book to be passed to a Custom GL Lines plug-in implementation. You can use this ID, for example, to load the accounting book record with the <code>nlapiLoadRecord(type, id, initializeValues)</code> SuiteScript API.

You can use this ID, for example, to define plug-in implementation functionality based on the accounting book ID, to load the record for the book with `nlapiLoadRecord(type, id, initializeValues)`, or to search for the record with `nlapiSearchRecord(type, id, filters, columns)`. See the help topic [SuiteScript 1.0 Documentation](#).

The type for an accounting book record is `accountingbook`.



Important: Limit the use of these APIs, where possible, to improve performance of the plug-in implementation. Also, searching for a record results in better performance than loading the record.

You can view the list of internal NetSuite IDs for accounting books at Setup > Accounting > Accounting Books.

You can also use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns int

Input Parameters None.

Parent object [AccountingBook](#)

Example

```

1  function customizeGllmpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var bookId = book.getId();
5      if (!book.isPrimary())
6      {
7          var bookRec = nlapiLoadRecord('accountingbook',bookId);
8          ...
9      }
10     ...
11 }
```

isPrimary()

Function Declaration boolean isPrimary()

Type Object method

Description Returns true if the book object is the primary accounting book for the NetSuite account or returns false if the accounting book is a secondary accounting book.

Returns boolean

Input Parameters None.

Parent object [AccountingBook](#)

Example

```

1  function customizeGllmpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var bookId = book.getId();
5      if (book.isPrimary())
6      {
7          var bookRec = nlapiLoadRecord('accountingbook',bookId);
8          ...
9      }
10     ...
11 }
```



```

9      }
10     ...
11    }

```

Record

Type Interface input object

Description Represents the current transaction record object passed to a Custom GL Lines plug-in implementation. Use the SuiteScript API functions available to the Record object to access field values on the transaction.



Note: You can only use functions to read values from the transaction record. You cannot write data to the transaction record.

Methods See the following table for details on the available methods and the help topic [nlobjRecord](#) in [SuiteScript 1.0 Documentation](#).

Parent Object(s) n/a

Child Object(s) n/a

Record Methods

Function	Description
<code>getAllFields()</code>	Returns a normal keyed array of all the fields on a record.
<code>getAllLineItemFields(group)</code>	Returns an array of all the field names of a sublist on the record.
<code>getFieldText(name)</code>	Returns the UI display value for a select field.
<code>getFieldTexts(name)</code>	Returns the UI display values for a multi-select field.
<code>getFieldValue(name)</code>	Returns the value (internal ID) of a field.
<code>getFieldValues(name)</code>	Returns the value (field ID) or values (array of field IDs) of a multi-select field.
<code>getId()</code>	Use this method to get the internal ID of a record when editing an existing transaction.
<code>getLineItemCount(group)</code>	Returns the number of lines on a sublist.
<code>getLineItemText(group, fldnam, linenum)</code>	Returns the display name of a select field (based on its current selection) in a sublist.
<code>getLineItemValue(group, name, linenum)</code>	Returns the value of a sublist line item field.
<code>getLineItemValues(type, fldname, linenum)</code>	Returns the values of a multiselect sublist field on a selected line.
<code>getRecordType()</code>	Returns the record type. For example, returns Unbuild for the Assembly Unbuild record type or returns SalesOrd for the Sales Order record type.
<code>viewLineItemSubrecord(sublist, fldname, linenum)</code>	Returns a <code>nlobjSubrecord</code> object. Use this API to view a subrecord from a sublist field on the parent record.

Function	Description
viewSubrecord(fldname)	Returns a nlobjSubrecord object. Use this API to view a subrecord from a body field on the parent record. NetSuite returns CANNOT_EDIT_SUBRECORD if you attempt to edit the subrecord returned by this function.

Common StandardLine and CustomLine Object Methods

The following object methods are referenced by both the [StandardLine](#) and [CustomLine](#) objects:

- [getAccountId\(\)](#)
- [getClassId\(\)](#)
- [getCreditAmount\(\)](#)
- [getDebitAmount\(\)](#)
- [getDepartmentId\(\)](#)
- [getEntityId\(\)](#)
- [getLocationId\(\)](#)
- [getMemo\(\)](#)
- [getSegmentValueId\(segmentId\)](#)

getAccountId()

Function Declaration	int getAccountId()
Type	Object method
Description	Returns the internal NetSuite account ID for a StandardLine or CustomLine object. This is the NetSuite general ledger account that gets credited or debited on the GL impact for a transaction. Use this ID to define plug-in implementation functionality based on the credited or debited account. See also setAccountId(accountId). You can view the internal NetSuite ID for all accounts on the Chart of Accounts page at Setup > Accounting > Chart of Accounts. You can use helper functions in a utility file so you don't have to add internal NetSuite IDs directly into the plug-in implementation logic. See Utility Files for a Custom GL Lines Plug-In Implementation.
Returns	int
Input Parameters	None.
Parent object	StandardLine , CustomLine

Example

```

1  function customizeGIImpact(record, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());

```

```

6      newLine.setAccountId(standardLines.getLine(0).getAccountId());
7      newLine.setMemo("Payment catches both revenue and cash.");
8      ...
9      var newLine = customLines.addNewLine();
10     newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
11     newLine.setAccountId(standardLines.getLine(1).getAccountId());
12     newLine.setMemo("Payment catches both revenue and cash.");
13     ...
14 }

```

getClassId()

Function Declaration `int getClassId()`

Type Object method

Description Returns the internal NetSuite ID for the class on a [StandardLine](#) or [CustomLine](#) object. Use this method to define plug-in implementation functionality based on the class for a GL impact line. See also [setClassId\(classId\)](#).

Classes are categories you can create to track records such as financials, transactions, and employees. You can view all classes defined in NetSuite at Setup > Company > Classes.

You can use helper functions in a utility file so you don't have to add internal NetSuite IDs directly into the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns `int`

Input Parameters None.

Parent object [StandardLine](#), [CustomLine](#)

Example

```

1      function customizeGLImpact(record, standardLines, customLines, book)
2      {
3          ...
4          var context = nlapiGetContext();
5          var makeClassesMandatory = context.getPreference('classmandatory');
6          var allowPerLineClasses = context.getPreference('classesperline');
7          ...
8          var newLine = customLines.addNewLine();
9
10         if (makeClassesMandatory || allowPerLineClasses)
11         {
12             // can also optionally set class
13             newLine.setClassId(standardLines.getLine(0).getClassId());
14         }
15         ...
16     }

```

getCreditAmount()

Function Declaration `string getCreditAmount()`

Type Object method

Description Returns the credit amount for a [StandardLine](#) or [CustomLine](#) object. Use this method to define plug-in implementation functionality based on the credited amount for a GL impact line. See also [setCreditAmount\(credit\)](#).

Returns `string`

Input Parameters None.

Parent object [StandardLine](#), [CustomLine](#)

Example

```

1  function customizeGLImpact(record, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());
6      newLine.setAccountId(standardLines.getLine(0).getAccountId());
7      newLine.setMemo("Payment catches both revenue and cash.");
8      ...
9      var newLine = customLines.addNewLine();
10     newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
11     newLine.setAccountId(standardLines.getLine(1).getAccountId());
12     newLine.setMemo("Payment catches both revenue and cash.");
13     ...
14 }

```

getDebitAmount()

Function Declaration string getDebitAmount()

Type Object method

Description Returns the debit amount for a [StandardLine](#) or [CustomLine](#) object. Use this method to define plug-in implementation functionality based on the debited amount for a GL impact line. See also [setDebitAmount\(debit\)](#).

Returns string

Input Parameters None.

Parent object [StandardLine](#), [CustomLine](#)

Example

```

1  function customizeGLImpact(record, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());
6      newLine.setAccountId(standardLines.getLine(0).getAccountId());
7      newLine.setMemo("Payment catches both revenue and cash.");
8      ...
9      var newLine = customLines.addNewLine();
10     newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
11     newLine.setAccountId(standardLines.getLine(1).getAccountId());
12     newLine.setMemo("Payment catches both revenue and cash.");
13     ...
14 }

```

getDepartmentId()

Function Declaration int getDepartmentId()

Type Object method

Description Returns the internal NetSuite ID for the department on a [StandardLine](#) or [CustomLine](#) object. Use this method to define plug-in implementation functionality based on the department for a GL impact line. See also [setDepartmentId\(departmentId\)](#).

Departments are listed first on transactions and are useful when designating transactions and employees as part of an internal team. You can view all departments defined in NetSuite at Setup > Company > Departments.

You can use helper functions in a utility file so you don't have to add the internal NetSuite IDs directly into the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns int

Input Parameters None.

Parent object [StandardLine](#), [CustomLine](#)

Example

```

1  function customizeGLImpact(record, standardLines, customLines, book)
2  {
3      ...
4      var context = nlapiGetContext();
5      var makeDepartmentsMandatory = context.getPreference('deptmandatory');
6      var allowPerLineDepartments = context.getPreference('deptsperline');
7      ...
8      var newLine = customLines.addNewLine();
9
10     if (makeDepartmentsMandatory || allowPerLineDepartments)
11     {
12         // can also optionally set department
13         newLine.setDepartmentId(standardLines.getLine(0).getDepartmentId());
14     }
15     ...
16 }

```

getEntityId()

Function Declaration int getEntityId()

Type Object method

Description Returns the internal NetSuite ID for the entity on a [StandardLine](#) or [CustomLine](#) object

Returns int

Input Parameters None

Parent object [StandardLine](#), [CustomLine](#)

Example

```

1  function customizeGLImpact(record, standardLines, customLines, book)
2  {
3      var entityId = standardLines.getLine(1).getEntityId();
4      var line = customLines.addNewLine()
5      line.setAccountId(6)
6      line.setDebitAmount(100)
7      line.setEntityId(entityId)
8
9      line = customLines.addNewLine()
10     line.setAccountId(7)
11     line.setCreditAmount(100)

```

12 | }

getLocationId()

Function Declaration int getLocationId()

Type Object method

Description Returns the internal NetSuite ID for the location on a [StandardLine](#) or [CustomLine](#) object. Use this method to define plug-in implementation functionality based on the location for a GL impact line. See also [setLocationId\(locationId\)](#).

Use locations to track information about employees and transactions for multiple offices or warehouses. You can view all locations defined in NetSuite at Setup > Company > Locations.

You can use helper functions in a utility file so you don't have to add the internal NetSuite IDs directly into the plug-in implementation logic. See [Utility Files for a Custom GL Lines Plug-In Implementation](#).

Returns int

Input Parameters None

Parent object [StandardLine](#), [CustomLine](#)

Example

```

1  function customizeGLImpact(record, standardLines, customLines, book)
2  {
3      ...
4      var context = nlapiGetContext();
5      var makeLocationsMandatory = context.getPreference('locmandatory');
6      var allowPerLineLocations = context.getPreference('locsperline');
7      ...
8      var newLine = customLines.addNewLine();
9
10     if (makeLocationsMandatory || allowPerLineLocations)
11     {
12         // can also optionally set location
13         newLine.setLocationId(standardLines.getLine(0).getLocationId());
14     }
15     ...
16 }
```

getMemo()

Function Declaration string getMemo()

Type Object method

Description Returns the **Memo** field on a [StandardLine](#) or [CustomLine](#) object. The **Memo** field comes from fields on the transaction, for example, the Description field for a line item on an invoice.

See also [setMemo\(memo\)](#).

Returns string

Input Parameters None.

Parent object [StandardLine](#), [CustomLine](#)

Example

```

1  function customizeGLImpact(record, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setCreditAmount(standardLines.getLine(0).getCreditAmount());
6      newLine.setAccountId(standardLines.getLine(0).getAccountId());
7      // append old memo to new memo text
8      newLine.setMemo("Payment catches both revenue and cash. " + standardLines.getLine(0).getMemo());
9      ...
10     var newLine = customLines.addNewLine();
11     newLine.setDebitAmount(standardLines.getLine(1).getDebitAmount());
12     newLine.setAccountId(standardLines.getLine(1).getAccountId());
13     // append old memo to new memo text
14     newLine.setMemo("Payment catches both revenue and cash. " + standardLines.getLine(1).getMemo());
15     ...
16     ...
17 }

```

getSegmentValueId(segmentId)

Function Declaration	int getSegmentValueId(segmentId)
Type	Object method
Description	Returns the internal NetSuite ID for the custom segment value set on the line on a StandardLine or CustomLine object. Use this method to define plug-in implementation functionality based on the value for a GL impact line. See also setSegmentValueId(segmentId, segmentValueId).
Returns	int
Input Parameters	segmentId {string} — String value of the custom segment ID
Parent object	StandardLine , CustomLine

Example

```

1  function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      for (var i = 0; i < standardLines.getCount(); i++)
5      {
6          var line = standardLines.getLine(i);
7
8          var costCenterId = line.getSegmentValueId('csegcostcenter');
9          var workCenterId = line.getSegmentValueId('csegworkcenter');
10
11         ...
12         // do something with the values
13     }
14
15     ...
16     var newLine = customLines.addNewLine();
17     // reading segment value from newly created line (value is null or copied from head)
18     var costCenterId = newLine.getSegmentValueId('csegcostcenter');
19     ...
20
21     var newLine2 = customLines.addNewLine();
22     ...
23     newLine2.setSegmentValueId('csegcostcenter', getCostCenterValueForItem(itemId));
24     if (newLine2.getSegmentValueId('csegcostcenter') == 1)
25     {
26         ...

```

```

27     }
28     ...
29 }
30 // Utility methods
31 function getCostCenterValueForItem(itemId)
32 {
33     ...
34     return valueId;
35 }

```

customizeGImpact(context)



Important: The customizeGImpact(context) function is used exclusively with SuiteScript 2.x, and is not compatible with SuiteScript 1.0

Function Declaration	function customizeGImpact(context)
Type	Interface function
Description	Contains the business logic to modify the general impact for a transaction with the Custom GL Lines Plug-in. Use the objects passed in the context to this function by NetSuite to implement the following business logic in a Custom GL Lines plug-in implementation: ■ Get information about general ledger impact. Use the StandardLines object to get information about each line of general ledger impact for a transaction. ■ Create custom lines. Use the CustomLines object to create lines with general ledger impact in primary and secondary accounts for a transaction. ■ Modify how GL impact in the primary book affects the secondary books. You can create different business logic for primary and secondary accounting books. See AccountingBook.
Returns	void
Input Parameters	context - A JavaScript object of CustomGLLinesPluginContext type with the following read-only properties: ■ StandardLines ■ CustomLines ■ ReadOnlyTransactionRecord ■ AccountingBook



Note: You can also use SuiteScript API functions to access transaction record properties in a plug-in implementation. See [Record](#).

standardLines

Type	StandardLines (read-only)
-------------	---

customLines

Type	CustomLines (read-only)
-------------	---

transactionRecord

Type	ReadOnlyTransactionRecord (read-only)
-------------	---

book

Type	AccountingBook (read-only)
------	----------------------------

Example

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType customgmlplugin
4   */
5  define(function() {
6      function customizeGImpact(context) {
7          var customLines = context.customLines;
8          var amount = '100.00';
9
10         var memo = context.transactionRecord.getValue({fieldId : 'memo'});
11         var firstAccount = context.standardLines.getLine({index: 0}).accountId;
12         var secondAccount = context.standardLines.getLine({index: 1}).accountId;
13         var bookId = context.book.id;
14
15         var firstLine = customLines.addNewLine();
16         firstLine.accountId = firstAccount;
17         firstLine.creditAmount = amount;
18         firstLine.memo = memo + ' for book ' + bookId;
19         firstLine.isBookSpecific = false;
20
21         var secondLine = customLines.addNewLine();
22         secondLine.accountId = secondAccount;
23         secondLine.debitAmount = amount;
24         secondLine.memo = memo + ' for book ' + bookId;
25         secondLine.isBookSpecific = false;
26     }
27
28     return {
29         customizeGImpact: customizeGImpact,
30     }
31 });

```

AccountingBook

Type	Object
Description	Represents the accounting book passed to a Custom GL Lines plug-in implementation when you save a transaction. Use the properties available to the book object to determine if the book is a primary or secondary book or get the internal NetSuite ID of the accounting book. If you use the Multi-Book Accounting feature, the AccountingBook object represents a different accounting book each time the plug-in implementation executes. For more information, see Custom GL Lines Plug-in Process Flow.
Properties	■ id ■ isPrimary

id

Property Description	Returns the internal NetSuite ID for the accounting book to be passed to a Custom GL Lines plug-in implementation. You can use this ID, for example, to define plug-in implementation functionality based on the accounting book ID, to load the record for the book with record.load(options), or to search for the record using the N/query Module.
----------------------	---

The type for an accounting book record is `accountingbook`.



Important: Limit the use of these APIs, where possible, to improve performance of the plug-in implementation. Also, searching for a record results in better performance than loading the record.

You can view the list of internal NetSuite IDs for accounting books at Setup > Accounting > Accounting Books.

You can also use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic.

Type	number (read-only)
Parent object	AccountingBook

isPrimary

Property Description	Returns true if the book object is the primary accounting book for the NetSuite account or returns false if the accounting book is a secondary accounting book.
Type	boolean (read-only)
Parent object	AccountingBook

CustomLines

Type	Object
Description	Contains the business logic to modify the general impact for a transaction with the Custom GL Lines Plug-in. Use the objects passed to this function by NetSuite to implement the following business logic in a Custom GL Lines plug-in implementation. ■ Get information about general ledger impact. Use the StandardLines object to get information about each line of general ledger impact for a transaction. ■ Create custom lines. Use the CustomLines object to create lines with general ledger impact in primary and secondary accounts for a transaction. ■ Modify how GL impact in the primary book affects the secondary books. You can create different business logic for primary and secondary accounting books. See AccountingBook. Contains an array of all custom lines with GL impact in a transaction as CustomLine objects. Use this object to add and modify custom lines with GL impact on a transaction. Create a new CustomLine object with addNewLine(). For more information, see Custom GL Lines Plug-in Process Flow.
Properties	count
Methods	■ addNewLine() ■ getLine(options)

count

Property Description	Returns the number of custom lines with GL impact for a specific accounting book in a transaction. These lines are added with addNewLine() . Use this property in conjunction
-----------------------------	---

with [getLine\(options\)](#) to get individual custom lines after you add them in the plug-in implementation.

Type	number (read-only)
Parent object	CustomLines

addNewLine()

Method Description	Adds a CustomLine object to the parent CustomLines object in a Custom GL Lines plug-in implementation and returns the new object. Use this method to add a custom line with GL impact to a transaction. After you create the custom line, use the methods available to the CustomLine object to set the properties of the custom line, including the general ledger account ID and the amount of the custom line.
--------------------	---



Note: If you use the Multi-Book Accounting feature and want this line to be copied to the secondary accounting books, use the [isBookSpecific](#) property.

Returns	CustomLine
Parent object	CustomLines

getLine(options)

Method Description	Returns a CustomLine object that represents a custom line with GL impact. CustomLine objects are stored in the CustomLines object starting at index 0.
Returns	CustomLine
Parent object	CustomLines

CustomLine

Type	Object
Description	Contains all properties for a single custom line for the GL impact on a transaction. Use the methods and properties available to the CustomLine object to set the values for the custom line and define plug-in implementation functionality based on the values. The CustomLines object contains a reference to each custom GL impact line. Create a new CustomLine object with addNewLine().
Properties	■ accountId ■ amount ■ classId ■ creditAmount ■ debitAmount ■ departmentId ■ entityId ■ isBookSpecific ■ locationId

	■ memo ■ segments
Methods	■ getSegmentValueId(options) ■ setSegmentValueId(options)
Parent Object(s)	CustomLines

accountId

Property Description	Sets the account ID property for a CustomLine object in a primary or secondary book. This value is the internal NetSuite ID for a general ledger account. You must use this property to set the account to be debited or credited for a custom line. You can view the internal NetSuite ID for all accounts on the Chart of Accounts page at Setup > Accounting > Chart of Accounts. You can use helper functions in a utility file to customize the internal NetSuite account IDs in the plug-in implementation logic.
Type	Object method
Parent object	CustomLine

amount

Property Description	Returns the amount for a CustomLine . If it is a credit, than the amount will include a negative sign.
Type	string (read-only)
Parent object	CustomLine

classId

Property Description	Sets the class ID value for a CustomLine object in a primary or secondary book. This value is the internal NetSuite ID for a class. Classes are categories that you can create to track records such as financials, transactions, and employees. You can view all classes defined in NetSuite at Setup > Company > Classes. See also classId. You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic. You can also access the runtime.User object with the N/runtime Module to get the preferences for classes at Setup > Accounting > Preferences > Accounting Preferences (Administrator).
Type	number - Internal NetSuite ID for a class
Parent object	CustomLine

creditAmount

Property Description	Sets the credit amount of a CustomLine object in a primary or secondary book. The value is rounded to currency precision.
-----------------------------	---

To set a debit amount, use the [debitAmount](#) property.



Note: The currency for the amount of the custom line depends on the subsidiary and accounting book to which the GL impact posts. For example, if the accounting book being processed by the plug-in implementation uses USD, the currency for the amount set by this method is USD.

Type string - The value of a credit on a general ledger account. This must be a positive value

Parent object [CustomLine](#)

debitAmount

Property Description Sets the debit amount of a [CustomLine](#) object in a primary or secondary book. The value is rounded to currency precision.

To set a credit amount, use the [creditAmount](#) property.



Note: The currency for the amount on the custom line depends on the accounting book to which the GL impact posts. For example, if the accounting book being processed by the plug-in implementation uses USD, the currency for the amount set by this method is USD.

Type string - The value of a credit on a general ledger account. This must be a positive value

Parent object [CustomLine](#)

departmentId

Property Description Sets the department ID for a [CustomLine](#) object in a primary or secondary book. This value is the internal NetSuite ID for a department.

Departments are listed first on transactions, and are useful when designating transactions and employees as part of an internal team. You can view all departments defined in NetSuite at Setup > Company > Departments. See also [departmentId](#).

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic. You can also access the [runtime.User](#) object with the [N/runtime Module](#) to get the preferences for locations at Setup > Accounting > Accounting Preferences.

Type Number - Internal NetSuite ID for a department.

Parent object [CustomLine](#)

entityId

Property Description Sets the entity ID property for a [CustomLine](#) object in a primary or secondary book.

Supported entity types include:

- Customers
- Jobs
- Leads
- Prospects

- Vendors
- Employees
- Other Names

The following restrictions apply to setting of entities on custom GL lines:

- Setting a vendor is not supported on lines with an Accounts Payable account type.
- Setting a customer is not supported on lines with an Accounts Receivable account type.
- Creating book-specific custom GL lines with an AP or AR account and entity combination. Only generic custom GL lines can be created with this combination.
- Only entities that have the same subsidiary as the transaction can be set on lines for that transaction.
- Only entities that have the same currency as the transaction can be set on lines for that transaction.
- Entities must be active.



Note: This property does not set the sales representative for the line, and it does not change the date of the first sale for the customer.

Type number - An internal NetSuite ID for an entity.

Parent object [CustomLine](#)

isBookSpecific

Property Description

Sets a custom GL impact line to affect only the primary book in a Custom GL plug-in implementation. If you use this property and set the value to `false`, the plug-in implementation copies the custom line to secondary books and the custom line is also subject to the mapping set up for the Multi-Book Accounting feature.

If you do not use this property or set the value to `true`, the custom GL impact line only applies to the primary accounting book and is not copied to secondary books.



Note: This property only applies if you use the Multi-Book Accounting feature and the property is being called for the primary accounting book. It has no impact on book-specific journal entries. For more information about how the plug-in implementation processes book-specific lines, see [Custom GL Lines Plug-in Process Flow](#).

Type boolean - Use `true` to make the custom line specific to the primary accounting book. Use `false` to direct NetSuite to copy the custom line to secondary accounting books.

Parent object [CustomLine](#)

locationId

Property Description Sets the location ID for a [CustomLine](#) object in a primary or secondary book. This value is the internal NetSuite ID for a location.

Use locations to track information about employees and transactions for multiple offices or warehouses. You can view all locations defined in NetSuite at Setup > Company > Locations. See also [locationId](#).

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic. You can also access the [runtime.User](#) object with the [N/runtime Module](#) to get the preferences for locations at Setup > Accounting > Accounting Preferences.

Type	number - Internal NetSuite ID for a location.
Parent object	CustomLine

memo

Property Description	Sets the Memo field on a CustomLine object.
Type	string - String text for the Memo field
Parent object	CustomLine

segments

Property Description	Returns a string array of available custom segment IDs for which a value can be set.
Type	string[] (read-only)
Parent object	CustomLine

getSegmentValueId(options)

Method Description	Returns the internal NetSuite ID for the custom segment value set on the line on a CustomLine object. Use this method to define plug-in implementation functionality based on the value for a GL impact line.
Returns	number
Parent object	CustomLine

Parameters

Parameter	Type	Required/Optional	Description
options.segmentId	String	Required	String value of the custom segment ID.

setSegmentValueId(options)

Method Description	Sets custom segment values on a CustomLine object NetSuite assigns custom segment values as follows: ■ If the custom segment has a body and lines, the value is copied from the body to all custom lines. The plug-in can be used to change the value of the lines. ■ If the custom segment has only a body value, the value is copied from the body to all custom lines. The plug-in can be used to change the value. ■ If the custom segment has only line values, no values are copied to custom lines.
Returns	Void
Parent object	CustomLine

Parameters

Parameter	Type	Required/Optional	Description
options.segmentId	string	Required	String value of the custom segment ID.
options.segmentValueId	number	Optional	Internal ID of the custom segment value that the custom line should be set to. Omitting this parameter unsets the custom segment value.

ReadOnlyTransactionRecord

Type	Object
Description	Represents the current transaction record object passed to a Custom GL Lines plug-in implementation. Use the SuiteScript API functions available to the ReadOnlyTransactionRecord object to access field values on the transaction. Field and sublist internal IDs can be found in the SuiteScript Record Browser. For more information, see the help topic Working with the SuiteScript Records Browser.

 **Note:** You can only use functions to read values from the transaction record. You cannot write data to the transaction record.

Properties	■ id ■ recordType ■ fields ■ sublists
Methods	■ findSublistLineWithValue(options) ■ getLineCount(options) ■ getSublistFields(options) ■ getSublistSubrecord(options) ■ getSublistText(options) ■ getSublistValue(options) ■ getSubrecord(options) ■ getText(options) ■ getValue(options) ■ hasSublistSubrecord(options) ■ hasSubrecord(options)
Child Object(s)	ReadOnlySubrecord

id

Property Description	Use this property to get the internal ID of a record when editing an existing transaction.
-----------------------------	--

 **Note:** With the asynchronous Custom GL Lines plug-in, id is set even when creating a transaction.

Type	number (read-only)
Parent Object	ReadOnlyTransactionRecord

recordType

Property Description	Returns the record type internal ID. For example, returns assemblyunbuild for the Assembly Unbuild record type or returns salesorder for the Sales Order record type.
Type	string (read-only)
Parent Object	ReadOnlyTransactionRecord

fields

Property Description	Returns the body field names (internal IDs) of all the fields in the transaction record, including the machine header field and matrix header fields.
Type	string[] (read-only)
Parent Object	ReadOnlyTransactionRecord

sublists

Property Description	Returns all the names (internal IDs) of all the sublists in the transaction record.
Type	string[] (read-only)
Parent Object	ReadOnlyTransactionRecord

findSublistLineWithValue(options)

Method Description	Returns the line number for the first occurrence of a field value in a sublist. Note that line indexing begins at 0 with SuiteScript 2.0.
Returns	A line number as a number, or -1 if not found.
Parent Object	ReadOnlyTransactionRecord

Parameters

Parameter	Type	Required/Optional	Description
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.
options.sublistId	string	Required	The internal ID of the sublist.
options.value	number date string array boolean	Optional	The value to search for.

getLineCount(options)

Method Description	Returns the number of lines in a sublist.
Returns	number

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.

getSublistFields(options)

Method Description Returns all the field names (internal IDs) in a sublist.

Returns string[]

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.

getSublistSubrecord(options)

Method Description Returns the subrecord associated with a sublist field.

Returns [ReadOnlySubrecord](#)

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.
options.line	number	Required	The line number for the field. Note that column indexing begins at 0 with SuiteScript 2.0.

getSublistText(options)

Method Description Returns the value of a sublist field in a text representation.



Important: This API is not available in the synchronous Custom GL Lines plug-in configuration.

Returns string

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.
options.line	number	Required	The line number for the field. Note that column indexing begins at 0 with SuiteScript 2.0.

getSublistValue(options)

Method Description	Returns the value of a sublist field.
Returns	number date string array boolean
Parent Object	ReadOnlyTransactionRecord

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.
options.line	number	Required	The line number for the field. Note that column indexing begins at 0 with SuiteScript 2.0.

getSubrecord(options)

Method Description	Returns the subrecord for the associated field.
Returns	ReadOnlySubrecord
Parent Object	ReadOnlyTransactionRecord

Parameters

Parameter	Type	Required/Optional	Description
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.

getText(options)

Method Description	Returns the text representation of a field value.
---------------------------	---



Important: This API is not available in the synchronous Custom GL Lines plug-in configuration.

Returns	string
----------------	--------

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.

getValue(options)

Method Description Returns the value of a field.

Returns number | date | string | array | boolean

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.

hasSublistSubrecord(options)

Method Description Returns the value indicating whether the associated sublist field contains a subrecord.

Returns boolean

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.
options.line	number	Required	The line number for the field. Note that column indexing begins at 0 with SuiteScript 2.0.

hasSubrecord(options)

Method Description Returns the value indicating whether the field contains a subrecord.

Returns boolean

Parent Object [ReadOnlyTransactionRecord](#)

Parameters

Parameter	Type	Required/Optional	Description
-----------	------	-------------------	-------------

options.fieldId string Required The internal ID of a standard or custom sublist field.

ReadOnlySubrecord

Type Object

Description Represents a subrecord object from the transaction record passed to a Custom GL Lines plug-in implementation. Use the SuiteScript API functions available to the ReadOnlySubrecord object to access field values on the subrecord.

Field and sublist internal IDs can be found in the SuiteScript Record Browser. For more information, see the help topic [Working with the SuiteScript Records Browser](#).



Note: You can only use functions to read values from the subrecord. You cannot write data to the subrecord.

Properties ■ [fields](#)
 ■ [sublists](#)

Methods ■ [findSublistLineWithValue\(options\)](#)
 ■ [getLineCount\(options\)](#)
 ■ [getSublistFields\(options\)](#)
 ■ [getSublistText\(options\)](#)
 ■ [getSublistValue\(options\)](#)
 ■ [getText\(options\)](#)
 ■ [getValue\(options\)](#)

Parent Object [ReadOnlyTransactionRecord](#)

fields

Property Description Returns the body field names (internal IDs) of all the fields in the subrecord, including the machine header field and matrix header fields.

Type string[] (read-only)

Parent Object [ReadOnlySubrecord](#)

sublists

Property Description Returns all the names (internal IDs) of all the sublists in the subrecord.

Type string[] (read-only)

Parent Object [ReadOnlySubrecord](#)

findSublistLineWithValue(options)

Method Description Returns the line number for the first occurrence of a field value in a sublist. Note that line indexing begins at 0 with SuiteScript 2.0.

Returns A line number as a number, or -1 if not found.

Parent Object [ReadOnlySubrecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.
options.value	number date string array boolean	Optional	The value to search for.

getLineCount(options)

Method Description Returns the number of lines in a sublist.

Returns number

Parent Object [ReadOnlySubrecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.

getSublistFields(options)

Method Description Returns all the field names (internal IDs) in a sublist.

Returns string[]

Parent Object [ReadOnlySubrecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.

getSublistText(options)

Method Description Returns the value of a sublist field in a text representation.



Important: This API is not available in synchronous Custom GL Lines plug-in configuration.

Returns string

Parent Object [ReadOnlySubrecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.sublistId	string	Required	The internal ID of the sublist.
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.
options.line	number	Required	The line number for the field. Note that column indexing begins at 0 with SuiteScript 2.0.

getText(options)

Method Description Returns the text representation of a field value.



Important: This API is not available in the synchronous Custom GL Lines plug-in configuration.

Returns string

Parent Object [ReadOnlySubrecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.

getValue(options)

Method Description Returns the value of a field.

Returns number | date | string | array | boolean

Parent Object [ReadOnlySubrecord](#)

Parameters

Parameter	Type	Required/Optional	Description
options.fieldId	string	Required	The internal ID of a standard or custom sublist field.

StandardLines

Type Object

Description Contains an array of all standard lines with GL impact in a transaction as StandardLine objects. Standard lines are the general ledger impacts that appear on the GL Impact report for a transaction.

Use this object to access individual standard lines for an accounting book in a transaction. For more information, see [Custom GL Lines Plug-in Process Flow](#).

Properties [count](#)

Child Object(s) [StandardLine](#)

count

Property Description	Returns the number of standard lines with GL impact for a specific accounting book in a transaction. Use this method in conjunction with to read individual standard lines. This property counts both visible and hidden lines on the GL Impact page. For an example of how to return only the number of visible lines, see Custom GL Lines Plug-in Guidelines and Best Practices .
Type	number (read-only)
Parent object	StandardLines

getLine(options)

Method Description	Returns a StandardLine object that represents a standard line with GL impact. StandardLine objects are stored in the StandardLines object starting at index 0.
Returns	StandardLine
Parent object	StandardLines

Parameters

Parameter	Type	Required/Optional	Description
options.index	number	Required	Index of the line to be returned by the method, starting at 0.

StandardLine

Type	Object
Description	Contains all values for a single standard line on the GL impact on a transaction. Use the methods and properties available to the StandardLine object to get the values for the standard line and define plug-in implementation functionality based on the values. The StandardLines object has a StandardLine object for each standard GL impact line.
Properties	■ accountId ■ amount ■ classId ■ creditAmount ■ debitAmount ■ departmentId ■ entityId ■ id ■ isPosting ■ isTaxable ■ locationId ■ memo ■ subsidiaryId ■ taxAmount ■ taxableAmount

	■ taxItemId ■ taxType
Methods	■ getSegmentValueId(options)
Parent Object	StandardLines

accountId

Property Description	Returns the internal NetSuite ID of the entity for a StandardLine object. This account ID represents the NetSuite general ledger account to which a credit or debit is applied on the GL impact for a transaction. Use this ID to define plug-in implementation functionality based on the credited or debited account. See also accountId. You can view the internal NetSuite ID for all accounts on the Chart of Accounts page at Setup > Accounting > Chart of Accounts. You can use helper functions in a utility file to customize the internal NetSuite account IDs in the plug-in implementation logic.
Type	number (read-only)
Parent object	StandardLine

amount

Property Description	Returns the amount for a StandardLine. Use this property to define plug-in implementation functionality based on the amount for a GL impact line. If the line is a credit, than it will have a negative sign.
Type	string
Parent object	StandardLine

classId

Property Description	Returns the internal NetSuite ID for the class on a StandardLine object. Use this property to define plug-in implementation functionality based on the class for a GL impact line. See also classId. Classes are categories that you can create to track records such as financials, transactions, and employees. You can view all classes defined in Setup > Company > Classes. You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic.
Type	number (read-only)
Parent object	StandardLine

creditAmount

Property Description	Returns the credit amount for a StandardLine object. Use this property to define plug-in implementation functionality based on the credited amount for a GL impact line. See also creditAmount.
Type	string (read-only)

Parent object [StandardLine](#)

debitAmount

Property Description Returns the debit amount for a [StandardLine](#) object. Use this property to define plug-in implementation functionality based on the credited amount for a GL impact line. See also [debitAmount](#).

Type string (read-only)

Parent object [StandardLine](#)

departmentId

Property Description Returns the internal NetSuite ID for the department on a [StandardLine](#) object. Use this property to define plug-in implementation functionality based on the class for a GL impact line. See also [departmentId](#).

Departments are categories that you can create to track records such as financials, transactions, and employees. You can view all classes defined in NetSuite at Setup > Company > Departments.

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic.

Type number (read-only)

Parent object [StandardLine](#)

entityId

Property Description Returns the internal NetSuite ID for the entity on a [StandardLine](#) object. The entity can be any customer, employee, partner, or vendor, depending on the type of transaction.

You can use this ID, for example, to define plug-in implementation functionality based on the entity ID, to load the record for the entity with [record.load\(options\)](#), or to search for the record using the [N/query Module](#) to look up the type of the entity.



Important: Limit the use of these APIs, where possible, to improve performance of the plug-in implementation. Also, searching for a record results in better performance than loading the record.

To implement functionality based on the entity ID, you can use helper functions to avoid adding the internal NetSuite ID into the plug-in implementation logic.

Type number (read-only)

Parent object [StandardLine](#)

id

Property Description Returns the internal NetSuite database ID for a standard GL impact line. The summary line for the GL impact on a transaction occurs for most transaction types with an ID of 0. Some transactions, like journal entries, do not have a summary line.

You can use the summary information to define plug-in implementation functionality based on the values in the summary line.



Note: The summary line for the GL impact on a transaction does not include tax information.

Type number (read-only)

Parent object [StandardLine](#)

isPosting

Property Description Returns true if the transaction is a posting transaction and the associated standard GL impact line posts to the general ledger. Returns false if the transaction is a non-posting transaction.

NetSuite transactions can be posting, non-posting, or posting with approval. For example, journal entries do not post without approval. For more information, see the help topic [General Ledger Impact of Transactions](#).

Type boolean (read-only)

Parent object [StandardLine](#)

isTaxable

Property Description Returns true if a standard GL impact line is a credit to a tax account. For example, if a standard line on a taxable transaction contains a credit value of 10.00 for an account of type Other Current Liability, this property returns true.

Type boolean (read-only)

Parent object [StandardLine](#)

locationId

Property Description Returns the internal NetSuite ID for the location on a [StandardLine](#) object. Use this property to define plug-in implementation functionality based on the location for a GL impact line. See also [locationId](#).

Use locations to track information about employees and transactions for multiple offices or warehouses. You can view all locations defined in NetSuite at Setup > Company > Locations.

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic.

Type number (read-only)

Parent object [StandardLine](#)

memo

Property Description Returns the **Memo** field on a [StandardLine](#) object. The **Memo** field is populated from fields on the transaction, for example, the Description field for a line item on an Invoice transaction.

See also [memo](#).

Type	string (read-only)
Parent object	StandardLine

subsidiaryId

Property Description Returns the internal NetSuite ID of the subsidiary for the entity associated with a standard GL impact line. You can view all subsidiaries and the associated internal IDs in NetSuite at Setup > Company > Subsidiaries.

You can use this ID, for example, to define plug-in implementation functionality based on the subsidiary ID, to load the record for the subsidiary with [record.load\(options\)](#), or to search for the record using the [N/query Module](#).



Important: Limit the use of these APIs, where possible, to improve performance of the plug-in implementation. Also, searching for a record results in better performance than loading the record.

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic.

Type	number (read-only)
Parent object	StandardLine

taxAmount

Property Description Returns a string that represents the amount of tax charged on a standard GL line. This property only returns a value other than 0 if the standard line is a credit to a tax account. For example, if a standard line on a taxable transaction contains a credit value of 10.00 for an account of type Other Current Liability, this property returns -10.

Use this property with [taxItemId](#), [taxableAmount](#), and [taxType](#) to get the tax information for a standard GL impact line.

Type	string (read-only)
Parent object	StandardLine

taxableAmount

Property Description Returns a string that represents the amount of a standard GL line that was subject to tax. Returns 0 if the line was not subject to tax. Use this property with [taxItemId](#), [taxAmount](#), and [taxType](#) to get the tax information for a standard GL impact line.

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic.

Type	string (read-only)
Parent object	StandardLine

taxItemId

Property Description Returns the internal NetSuite ID of the tax code for a standard GL line. Returns null if the line was not subject to tax or if the line does not credit a tax account with a tax liability.

You can use this ID to define plug-in implementation functionality based on the tax code. For example, a tax code of -7 indicates the line was not subject to tax. You can view all tax codes in NetSuite at Setup > Accounting > Tax Codes.

You can use helper functions in a utility file to avoid adding the internal NetSuite IDs into the plug-in implementation logic.

Use this property with [taxableAmount](#), [taxAmount](#), and [taxType](#) to get the tax information for a standard GL impact line.

Type	number (read-only)
Parent object	StandardLine

taxType

Property Description	Returns the tax type for a standard GL line that was subject to tax. Returns null if the standard line was not subject to tax. You can use this ID to define plug-in implementation functionality based on the tax type. You can view all tax types in NetSuite at Setup > Accounting > Tax Types. Use this property with taxableAmount, taxAmount, and taxItemId to get the tax information for a standard GL impact line.
Type	string (read-only)
Parent object	StandardLine

getSegmentValueId(options)

Method Description	Returns the internal NetSuite ID for the custom segment value set on the line on a StandardLine object. Use this method to define plug-in implementation functionality based on the value for a GL impact line. See also getSegmentValueId(options) and setSegmentValueId(options).
Returns	number
Parent object	StandardLine

Parameters

Parameter	Type	Required/Optional	Description
options.segmentId	String	Required	String value of the custom segment ID.

Custom GL Lines Plug-in Reference

Use the information in this section as reference when you develop, run, and test a Custom GL Lines plug-in implementation.

Topic	Description
View example code	View example code for using utility files in a plug-in implementation. Use the utility files to create helper functions, reference internal NetSuite IDs, search records, and store internal NetSuite record IDs using a custom record. See Custom GL Lines Plug-in Example Code .
Classification handling	You can set classifications, including department, class, and location, on custom lines for standard transactions, journals, and custom transactions. Depending on the accounting preferences, NetSuite may set the classifications. See Classifications .
Guidelines and best practices	Get more information about creating the plug-in implementation script file and plug-in implementation behavior. See Custom GL Lines Plug-in Guidelines and Best Practices .
Get help on error messages	Get more information about error messages that can occur when NetSuite validates the plug-in implementation logic or when the plug-in implementation script runs. See Error Messages for Custom GL Lines Plug-in .
Searching and reporting on custom lines	Use the Custom GL flag on reports and searches. See Custom Lines on Reports and in Searches .
Use the audit log	Use the Custom GL Lines audit log to learn more about custom lines created by a plug-in implementation. See Custom GL Lines Audit Log .
Use the plug-in system notes	Use the plug-in system notes to learn more about changes made on a plug-in implementation record. See the help topic Viewing Plug-in Implementation System Notes .
Delete a plug-in implementation	Delete a Custom GL Lines plug-in from a NetSuite account, or uninstall a bundle that contains a plug-in implementation. See Deleting a Custom GL Lines Plug-in Implementation .

Custom GL Lines Plug-in Example Code

Use the code examples in this section as a reference when you create a Custom GL Lines plug-in implementation.

The following table describes the examples.

Example	Description
Using utility files	Use utility JavaScript files to include helper functions for the main plug-in implementation script file or store internal NetSuite IDs for NetSuite records. See Utility Files for a Custom GL Lines Plug-In Implementation .

Example	Description
Using a custom record	Use a custom record to store references for required NetSuite standard records and then retrieve them using SuiteScript. See Using a Custom Record to Reference Internal NetSuite IDs Example .

Utility Files for a Custom GL Lines Plug-In Implementation

You can create utility JavaScript files for use with a Custom GL Lines plug-in implementation. These utility files can contain the following types of logic:

- Helper functions for use with the main plug-in implementation script file. See [Helper Functions Example](#).
- Internal NetSuite IDs for NetSuite records. See [Internal NetSuite ID Example](#).

Depending on your plug-in implementation requirements, you may want to avoid using internal NetSuite IDs in the implementation and utility script files. In this case, you can create a custom record type to store references to the internal IDs for NetSuite records, and then retrieve the IDs through SuiteScript. For an example, see [Using a Custom Record to Reference Internal NetSuite IDs Example](#).

Helper Functions Example

You can use helper functions similar to the examples below to avoid using NetSuite internal IDs in the plug-in implementation logic.

Utility Javascript File

```

1  // search for entity's email
2  function findEntityEmail(entityId)
3  {
4      var searchFilters = new Array();
5      searchFilters[0] = new nlobjSearchFilter('internalid', null, 'anyof', entityId);
6      var searchColumns = new Array();
7      searchColumns[0] = new nlobjSearchColumn('email');
8
9      var searchResults = nlapiSearchRecord('entity', null, searchFilters, searchColumns);
10     var email = '';
11     if (searchResults != null && searchResults.length > 0)
12     {
13         var email = searchResults[0].getValue('email');
14     }
15     return email;
16 }
17
18 // search for entity's phone with saved search
19 function findEntityPhone(entityId)
20 {
21     var searchFilters = new Array();
22     searchFilters[0] = new nlobjSearchFilter('internalid', null, 'anyof', entityId);
23
24     var savedSearchResults = nlapiSearchRecord('entity', 'customsearch_script_id', searchFilters);
25     var phone = '';
26     if (savedSearchResults != null && savedSearchResults.length > 0)
27     {
28         phone = savedSearchResults[0].getValue('phone');
29     }
30 }

```

Plug-in Implementation Script File

```

1  function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2  {

```

```

3      ...
4      var entityId = transactionRecord.getFieldValue('entity');
5      var email = findEntityEmail(entityId);
6      var phone = findEntityPhone(entityId);
7      ...
8  }

```

Internal NetSuite ID Example

The interface for the Custom GL plug-in includes methods that you can use to retrieve the internal NetSuite IDs for NetSuite records related to the transaction. You can use these IDs to define plug-in implementation functionality or with SuiteScript APIs like for example, `nlapiLoadRecord(type, id, initializeValues)` and `nlapiSearchRecord(type, id, filters, columns)`. For more information, see [SuiteScript 1.0 API Reference](#). However, these IDs are specific to a particular NetSuite account. You may want to include the account-specific IDs in a separate utility file so that they may be later customized if the plug-in implementation is installed in a different account.

The following example shows a helper function named `getAccountIDforCustomLine(accountId)` that returns the internal NetSuite ID for an account record. The account ID to return is included in the `getAccountIDforCustomLine(accountId)` function. The function is referenced in the main plug-in implementation script file.

Note: This code is for demonstration purposes only. The `customizeGImpact(transactionRecord, standardLines, customLines, book)` function does not add balanced custom lines and the transaction will not save.

Utility Javascript File

```

1  // Utility methods
2  function getAccountIDforCustomLine(accountId)
3  {
4      ...
5      if (accountId == 30) // main transaction account is 4000 - Sales (id: 30)
6      {
7          return 40; // custom line account is 1000 - VAT Expenses (id:40)
8      } else {
9          // return something else
10         ...
11     }
12 }

```

Plug-in Implementation Script File

```

1  function customizeGImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var newLine = customLines.addNewLine();
5      newLine.setDebitAmount(amount);
6      newLine.setAccountId(getAccountIDforCustomLine(standardLines.getLine(0).getAccountId()));
7      newLine.setMemo("Payment catches both revenue and cash.");
8      ...
9  }

```

Using a Custom Record to Reference Internal NetSuite IDs Example

You can use a custom record type to store the references to the required NetSuite standard records and retrieve them through SuiteScript. For more information about creating custom record types, see the help topic [Creating Custom Record Types](#) in the Customization documentation.

Note: The following custom record example and plug-in implementation code is for demonstration purposes only. Your implementation may differ depending on your requirements. You can use a similar solution to retrieve other IDs, like subsidiaries or entities. Also, you use a similar method to create a solution that is not language-dependent.

To store and retrieve references to internal NetSuite IDs, complete the following steps:

1. Create a custom record type with an ID of **customrecord_tutorial_account_config** and add a field that references a standard NetSuite record. For example, create a custom record type named **Account Configuration** to store references to the accounts required by the plug-in implementation. Then, create a field for the custom record type with the following configuration:

Field Property	Value
Label	Account
ID	_account
Type	List/Record
List/Record	Account

2. Create a new instance of the custom record. For this example, you create a custom record with a name of **Cash Basis Income Account** that references a NetSuite account that you want to use with a custom line in the plug-in implementation.
3. Create the plug-in implementation and utility script files. The helper function `findConfiguredAccount(name)` searches for the account name and returns the internal NetSuite account ID.

Utility JavaScript File

```

1 // Returns internal ID of account from a custom record with the name specified in parameter
2 function findConfiguredAccount(name)
3 {
4     var searchFilters = new Array();
5     searchFilters[0] = new nlobjSearchFilter('name', null, 'is', name);
6     searchFilters[1] = new nlobjSearchFilter('isinactive', null, 'is', 'F');
7
8     var searchColumns = new Array();
9     searchColumns[0] = new nlobjSearchColumn('custrecord_account');
10
11     var searchResults = nlapiSearchRecord('customrecord_tutorial_account_config', null, searchFilters, searchColumns);
12
13     if (searchResults!=null && searchResults.length > 0)
14     {
15         return searchResults[0].getValue('custrecord_account');
16     }
17     throw "Account " + name + " not found";
18 }

```

Plug-in Implementation Script File

```

1 function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2 {
3     ...
4     var accountId = findConfiguredAccount('Cash Basis Income Account');
5     line.setAccountId(accountId);
6     ...
7 }

```

Classifications

You can use the [CustomLine](#) methods in a Custom GL Lines plug-in implementation to set department, class, and location classifications on custom lines. If you do not set classifications on custom lines, NetSuite may set the classifications, depending on the accounting preferences or the transaction form.

Refer to the following sections for information about how NetSuite handles classifications for the following types of transactions:

- **Standard transactions.** Classifications are set on custom lines depending on the plug-in implementation behavior and the accounting preferences. See [Classification Handling on Standard Transactions](#).
- **Journals.** In addition to the classification handling on standard transactions, NetSuite also respects the journal-specific accounting preferences when setting classifications on custom lines. See [Classification Handling on Journals](#) and [Classification Handling on Standard Transactions](#).
- **Custom transactions.** NetSuite uses the preferences set on the custom transaction definition when setting classifications on custom lines. See [Classification Handling on Custom Transactions](#).

Classification Handling on Standard Transactions

Setting department, class, and location classifications on custom lines for standard transactions depends on the plug-in implementation behavior and the accounting preferences.

For example, if classifications are required, all custom lines inherit header classifications. Otherwise, if classifications can be set at the line level, the plug-in implementation can set the classifications. If the plug-in implementation does not set classifications, the classifications from the transaction header are applied to any custom line.



Important: These rules apply only for standard transactions. Custom transactions use the preferences set on the custom transaction definition. See [Classification Handling on Custom Transactions](#).

For standard transactions, setting classifications on custom lines by NetSuite and the plug-in implementation depends on the following accounting preferences:

- Mandatory classifications and per-line classification preferences. You can set preferences that make the department, class, and location classification required on transactions or set preferences that allow classifications to be set at the line level. See the help topics [General Accounting Preferences](#) and [Using Per-Line Classifications](#).
- Level at which classifications display on transaction forms. If you have set preferences to use per-line classes, departments, or locations, you can also customize transaction forms to identify and show the classification at both the header and line level at the same time. See the help topic [Customizing Forms for Per-Line and Header Classifications](#).

The following table describes how the department, class, and location classification handling depends on the accounting preferences:

Show Classification Level	Allow Classifications at Line- Level	Plug-in Implementation Sets Classification	Result on Custom Line
Header	No	Yes	Use classification from transaction header.*
Header	No	No	Use classification from transaction header.

Show Classification Level	Allow Classifications at Line- Level	Plug-in Implementation Sets Classification	Result on Custom Line
Header	Yes	Yes	Use classification set by plug-in implementation.
Header	Yes	No	Use classification from transaction header.
Line	Yes	Yes	Use classification set by plug-in implementation.
Line	Yes	No	No classification.
None	No	Yes	No classification.
None	No	No	No classification.
None	Yes	Yes	Use classification set by plug-in implementation.
None	Yes	No	No classification.

* NetSuite overrides the plug-in implementation classification setting.

Guidelines for Classifications Handling

Use the following guidelines when working with classifications for standard transactions and a Custom GL Lines plug-in implementation:

- The preceding classification handling applies only to custom lines. Standard lines use the value as determined by the accounting preferences and the values entered on the transaction form.
- If you set the preference to make a classification required and use a form customized to display a classification at both the header and line level, the classification is required at both the header and line level.

Classification Handling on Journals

The following table lists the journal-specific accounting preferences and classification handling on custom lines.

Preference	Classification Handling
Always Allow Per-line Classifications on Journals	Set line-item classes, departments, and locations on custom lines for journal entries. Overrides any other setting for standard transaction classifications. See Classification Handling on Standard Transactions and General Accounting Preferences .
Allow Non-balancing Classifications on Journal Entries	Set non-balancing classifications on custom lines for a journal. See the help topic General Accounting Preferences . For example, if you set a debit amount for a custom line with a specific classification, the transaction does not require a balancing credit for the same classification.
Allow Empty Classifications on Journals	Journal line-items do not contain classes, departments, and locations.

Preference	Classification Handling
	Overrides any other setting for standard transaction classifications. See Classification Handling on Standard Transactions and General Accounting Preferences .

Classification Handling on Custom Transactions

The classifications that you can set on custom lines for a custom transaction depend on the configuration of the custom transaction type.

You can set the following preferences for departments, classes, and locations on a custom transaction:

Preference	Custom Lines Handling
None	No value is required for the department, class, or location on instances of the custom transaction.
Header	Specify the classification at the header level for instances of the custom transaction. All custom lines inherit this value. If you use a CustomLine method to set the classification value, NetSuite ignores the value and uses the header value on all custom lines.
Line	Specify the classification at the line-item level for all lines on the custom transaction. You must use a CustomLine method to set the value of the classification on each custom line created by the plug-in implementation. Otherwise, you cannot save the transaction.

The classification settings on a custom transaction override any general accounting preferences you specify at Setup > Accounting > Accounting Preferences. For example, you make the department classification required for transactions in the general accounting preferences, but set the **Department** option on a custom transaction type to **None**. Consequently, the **Department** classification is not required for instances of the custom transaction, overriding the general accounting preferences.

Multiple Currencies Examples

When you're working with custom lines in an account that has multiple accounting books or currencies, NetSuite automatically handles currency conversion in the following situations:

- If the transaction currency is different from the accounting book currency where the GL impact posts, NetSuite converts standard lines to the base currency. Custom lines don't need conversion so NetSuite creates the custom lines using the base currency.

For example, if a transaction uses EURO, but the accounting book uses GBP, NetSuite converts from EURO to GBP when calculating the GL impact, based on the exchange rate in NetSuite or the one set on the transaction.

For more information, see the help topic [Currency Management](#).
- NetSuite maps custom lines from a primary accounting book to a secondary one, and the two accounting books use different currencies.

For example, if the primary accounting book uses USD and the secondary book uses GBP, and you use [setCreditAmount\(credit\)](#) to set a custom line on the primary book with [setBookSpecific\(bookSpecific\)](#) set to false.

NetSuite converts the amount from USD to GBP when it adds the custom line impact to the secondary book. For more information about how lines are mapped between primary and secondary accounting books in a Custom GL Lines plug-in implementation, see [Custom GL Lines Plug-in Process Flow](#).

The currency NetSuite converts to depends on both the subsidiary and the accounting book currency. For example, a US company has two subsidiaries:

Subsidiary Name	Secondary Book 1 Currency	Secondary Book 2 Currency
United States	USD	GBP
Canada	CAD	Euro

Any GL impact posts to the secondary accounting books based on the currency for the subsidiary accounting book. The GL impact shows in the currency for the accounting book, not the one on the transaction line items.

Converting Currency in a Custom GL Lines Plug-in Implementation

In general, you don't need to manually convert currencies if you're using the amounts of standard lines posted to a transaction to calculate the values of custom lines. However, if you want to use hard-coded currency values or retrieve an amount for a custom line from somewhere other than a standard GL impact line, you need to manually convert the currency amounts.

To perform the conversion, complete the following steps:

1. Get the base currency. You can search for the base currency or get it from the accounting book record. See [Find the Base Currency](#) and [Base Currency from Accounting Book Record](#).
2. Perform the conversion. Get the transaction-specific conversion rate from the accounting book record or use the `nlapiExchangeRate` API. See [Transaction-Specific Exchange Rates](#) or [Convert Currencies with SuiteScript](#).

Find the Base Currency

Use the following function to find the base currency for the current accounting book using `nlapiSearchRecord(type, id, filters, columns)`. For more information, see [SuiteScript 1.0 API Reference](#).

```

1  /* Returns internal id of base currency for specified subsidiary and accounting book combination */
2  function getBaseCurrency(subsidiaryId, bookId)
3  {
4
5      var searchFilters = new Array();
6      searchFilters[0] = new nlobjSearchFilter('internalid', null, 'is', subsidiaryId);
7      searchFilters[1] = new nlobjSearchFilter('accountingbook', null, 'is', bookId);
8
9      var searchColumns = new Array();
10     searchColumns[0] = new nlobjSearchColumn('accountingbookcurrency');
11
12     var searchResults = nlapiSearchRecord('subsidiary', null, searchFilters, searchColumns);
13
14     if (searchResults != null && searchResults.length > 0)
15     {
16         return searchResults[0].getValue('accountingbookcurrency');
17     }
18
19     throw "Subsidiary " + subsidiaryId + " or accounting book " + bookId + " not found";
20 }

```

Base Currency from Accounting Book Record

Depending on the transaction type, you can get the base currency from the transaction record. For example, a Cash Sale record contains information about the accounting book and the associated base currency for the transaction:

Items	Shipping	Billing	Accounting	Relationships	Communication	Related Records	System Information	Custom	Charges	Accounting Books	GL Impact	GL Matching
Secondary Book	Base Currency		Exchange Rate	Rev Rec on Rev Commit.		Transaction Is VSOE Bundle						
IFRS book	British pound		0.02936963	No		No						
Local book	Czech crown		1.00	No		No						

Use the following example code to find the base currency for the current accounting book using the [Record](#) object:

```

1  function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var bookId = book.getId();
5      var baseCurrency;
6
7      for (var line = 1; line <= transactionRecord.getLineItemCount('accountingbookdetail'); line++)
8      {
9          if (bookId == transactionRecord.getLineItemValue('accountingbookdetail', 'bookid', line))
10         {
11             baseCurrency = transactionRecord.getLineItemValue('accountingbookdetail', 'currency', line);
12
13             break;
14         }
15     }
16     ...
17 }

```

Transaction-Specific Exchange Rates

If a user can define transaction-specific exchange rates for the accounting book on a transaction, you can use the conversion rate for the accounting book to perform the conversion.

The following screenshot shows the **Exchange Rate** on the **Accounting Books** subtab of a Cash Sale transaction:

Items	Shipping	Billing	Accounting	Relationships	Communication	Related Records	System Information	Custom	Charges	Accounting Books	GL Impact	GL Matching
Secondary Book	Base Currency		Exchange Rate	Rev Rec on Rev Commit.		Transaction Is VSOE Bundle						
IFRS book	British pound		0.02936963	No		No						
Local book	Czech crown		1.00	No		No						

Use the following example code to find the exchange rate by accessing the exchange through the [Record](#) object:

```

1  function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var amountToAdd = 20; // Amount in Primary book's currency
5
6      if (!book.isPrimary()) // Amount in Primary book's currency, convert only for non-primary books
7      {
8          var bookId = book.getId();
9
10         for (var line = 1; line <= transactionRecord.getLineItemCount('accountingbookdetail'); line++)
11         {
12             if (bookId == transactionRecord.getLineItemValue('accountingbookdetail', 'bookid', line))
13             {
14                 var rate = transactionRecord.getLineItemValue('accountingbookdetail', 'exchangerate', line);
15                 amountToAdd = amountToAdd * rate;
16
17                 break;
18             }
19         }
20     }
21     var newLine = customLines.addNewLine();

```

```

22     newLine.setDebitAmount(amountToAdd);
23     ...
24 }

```

Convert Currencies with SuiteScript

The **Exchange Rate** column of the **Currency Exchange Rates** record displays the company-wide exchange rates. To access the Currency Exchange Rates record, go to Lists > Accounting > Currencies.

You can access these values with the `nlapiExchangeRate(sourceCurrency, targetCurrency, effectiveDate)` API. For more information, see [📄 SuiteScript 1.0 API Reference](#).

Use the following example code to perform a conversion using the company-wide exchange rates:

```

1  function customizeGLImpact(transactionRecord, standardLines, customLines, book)
2  {
3      ...
4      var amountToAdd = 20; // USD
5      var amountCurrency = 1; // internal id of USD currency in their account
6      var subsidId = transactionRecord.getFieldValue('subsidary');
7      var bookId = book.getId();
8      var targetCurrency = getBaseCurrency(subsidId, bookId);
9
10     if(targetCurrency!=amountCurrency)
11     {
12         var rate = nlapiExchangeRate(amountCurrency, targetCurrency);
13
14         amountToAdd = amountToAdd * rate;
15     }
16
17     var newLine = customLines.addNewLine();
18
19     newLine.setDebitAmount(amountToAdd);
20     ...
21 }

```

Custom GL Lines Plug-in Guidelines and Best Practices

The following table describes guidelines and best practices for working with the Custom GL Lines plug-in:

Guidelines	Description
Verify environment	Perform the following checks to verify the environment and define plug-in implementation behavior based on the results: - Transaction type. Check the transaction type. Use <code>getRecordType()</code>. For more information, see 📄 SuiteScript 1.0 API Reference. For an example, see the example for <code>getEntityId()</code>. - Multi-Book Accounting feature and primary book. Check if the Multi-Book Accounting feature is enabled and if the current book processed by the plug-in implementation is the primary accounting book. See <code>isPrimary()</code>. - Classification settings. Check the accounting preferences for department, class, and location in the current account. Use <code>nlapiGetContext()</code> (see 📄 SuiteScript 1.0 API Reference) and <code>getDepartmentId()</code>, <code>getClassId()</code>, or <code>getLocationId()</code>. You can use this information to define the plug-in behavior for the types of transactions configured for the implementation and how the custom lines should be created.

Guidelines	Description
Account and amount	At minimum, each custom line created by the plug-in implementation must contain an account ID and an amount for the custom line. See setAccountId(accountId) and one of the following methods: setCreditAmount(credit) or setDebitAmount(debit) .
Utility files	Use utility files to include helper functions that assist in defining plug-in implementation behavior. See Utility Files for a Custom GL Lines Plug-In Implementation .
Validations	NetSuite performs the following validations on the logic in the plug-in implementation script file when the script runs: - Input to methods that set the custom line amount is numerical. - Script file accesses standard lines that exist. - Each custom line contains an amount and an account ID. - Statistical account IDs are not used for account IDs. - IDs for accounts, departments, classes, and locations exist in the current NetSuite account. - Debits and credits on the GL impact lines balance. - Currency type for the account matches the base currency for the subsidiary entity or matches the currency for the transaction.
Transaction must balance	The total amount of debits and credits for the transaction must balance. Each plug-in implementation only runs on the subsidiary for the current entity. You cannot distribute credits and debits across subsidiaries.
Using the main line	The first line in the StandardLines object, Line 0, is a summary line that contains the total amount of credits and debits for the transaction. The account ID for the main line depends on the transaction properties, line items, and accounting book. You can use this account ID when setting the account ID for a new custom line.
Creating new versions	To update an existing, installed version of a plug-in implementation, you should create a new implementation of the plug-in with a different name, instead of pushing an update to the existing plug-in implementation.
Searching instead of loading records	You can use SuiteScript API functions in a plug-in implementation, for example, nlapiLoadRecordType(type, id, initializeValues) and nlapiSearchRecord(type, id, filters, columns). For more information, see SuiteScript 1.0 API Reference. However, using these APIs may negatively affect the performance of the plug-in implementation. Limit the use of these APIs, where possible, to improve performance of the plug-in implementation. In general, searching for records yields better performance than loading NetSuite records.
Creating records or performing data processing	You should limit the logic of a plug-in implementation to the features required to create custom lines on the GL impact. Additional functionality, for example, creating records or performing other complex data processing operations, should not be added to the plug-in implementation script file.
Using the <code>getCount()</code> method to return the number of visible lines	By default, the getCount() method returns the number of visible and hidden lines on the GL Impact page. If you want to find only the number of visible lines, use the following condition. <pre> 1 if((debitAmount !=0 creditAmount !=0) && accountId != null) { 2 // normal visible lines </pre>

Guidelines	Description
	<pre>3 }</pre> Or use the following shorter version. <pre>1 if((debitAmount creditAmount) && accountId) { 2 // normal visible lines 3 }</pre>

Governance Limit

The plug-in script file allows up to 1000 usage units. For more information, see the help topic [Script Type Usage Unit Limits](#).

Error Messages for Custom GL Lines Plug-in

Errors can occur when NetSuite validates the plug-in implementation logic before the script runs or during the plug-in implementation script process. See the following sections for assistance in resolving error messages related to the custom GL lines plug-in.

- [Invalid Operation Error Messages](#)
- [General Output Validation Error Messages](#)
- [Validation Error Messages for Empty Values](#)
- [Validation Error Messages for Invalid Values](#)

Invalid Operation Error Messages

All of the following error messages are preceded by: **Custom GL Lines Plugin error due to an invalid script operation.**

Error Message	Cause	Preferred Action
Cannot Parse Value: {amount}	You used setDebitAmount(debit) or setCreditAmount(credit) to add an amount to a custom line but the amount is either NULL or not a number.	Verify that the parameter value for the method is a non-null number in the plug-in implementation script file or utility file.
Cannot use {amount} as input to setDebitAmount(debit). Amount to debit must be positive	You used setDebitAmount(debit) to add a debit amount to a custom line but the amount is negative.	Use a positive value for setDebitAmount(debit) .
Line with index {index} does not exist. Total line count is: {lineCount}	You used getLine(index) to get a standard or custom line. However, the index value is equal to or greater than the total number of lines in the StandardLine or CustomLine object.	Make sure the index parameter for getLine(index) references a valid element. You can use getCount() to get the total number of lines.
Cannot use {amount} as input to setCreditAmount(credit). Amount to credit must be positive	You used setCreditAmount(credit) to add a credit amount to a custom line, but the amount is negative.	Use a positive value for setCreditAmount(credit) .

General Output Validation Error Messages

All of the following error messages are preceded by: **Custom GL Lines Plugin error due to a failed validation of script output:**

Error Message	Cause	Preferred Action
Statistical account {id} was used in financial journal entry. Statistical accounts are not allowed in financial journal entries	The transaction is a Journal and you created a new CustomLine object with addNewLine() . However, you used a statistical account ID for setAccountId(accountId) .	Make sure the ID for the account isn't for a statistical account. Statistical accounts appear on the Chart of Accounts page at Setup > Accounting > Chart of Accounts with a type of Statistical .
Transaction was not in balance. Total = {amount}	A Custom GL Lines plug-in implementation creates custom lines for a transaction, but the total debits and credits for the custom lines in the transaction don't balance.	Make sure that the plug-in implementation logic creates custom lines with equal credits and debits.
Transaction and foreign currency account use different currencies. Account internal ID: {id}	You used setCreditAmount(credit) or setDebitAmount(debit) to add an amount to a custom line. However, the currency for the account doesn't match either the base currency for the subsidiary entity or the currency for the transaction.	Choose an account with a currency that matches the base currency for the subsidiary entity or the currency for the transaction.
{classification types} are required on all lines of the journal entry	You didn't include a required classification type on every line of the journal entry.	On the Accounting Preferences General page, determine whether the Make Departments Mandatory, Make Classes Mandatory, and Make Locations Mandatory boxes are checked. If the boxes are checked, you must provide the classification types on every line of the journal entry.
The role set up to run the plugin does not have permissions to set values on the custom segment {script ID of custom segment}	The role of the user who is running the plug-in doesn't have permission to set values on the custom segment.	Provide the user the permissions needed to set values on the custom segment. For more information, see the help topic Granting a Role Permission to Manage Custom Segment Values .
Setting entity of type {entity type} on the same GL line with account of type {account type} is not allowed	You specified an account type and entity type combination that isn't allowed. For example, you specified a vendor with an accounts payable account, or a customer with an accounts receivable account.	Ensure that the account type specified in setAccountId(accountId) is appropriate for the entity type
Setting entity on GL line with account of type {account type} is not allowed for book-specific transactions	You specified an entity with an accounts receivable or accounts payable account type that's not allowed with a book-specific line.	Ensure that the account type specified in setAccountId(accountId) allows book-specific transactions.
You have entered entity IDs belonging to these entity types: {entity type}. These entity types	You specified an entity ID whose entity types aren't supported on custom GL lines.	Enter an entity ID in setEntityId(entityId) that belongs to supported entity types.

Error Message	Cause	Preferred Action
are not supported on custom GL lines.		
Entity {entity id} and transaction use different currencies	The transaction and entity use different currencies.	Specify an entity in setEntityId(entityId) that uses the same currency as the transaction. For more information, see Multiple Currencies Examples .
Setting inactive entity {entity id} on custom GL lines is not allowed	You specified an inactive entity ID on the custom GL line.	Specify a valid entity in setEntityId(entityId) .

Validation Error Messages for Invalid Values

All of the following error messages are preceded by: **Custom GL Lines Plugin error due to a failed validation of script output:**

Error Message	Cause	Preferred Action
Custom segment {script ID of custom segment} is invalid. Possible causes:	The custom segment is invalid. The error message lists all the possible causes.	Ensure that the custom segment exists, and verify the settings listed in the error message. For more information, see the following topics.
■ The Custom Segments feature is not enabled ■ Custom segment does not exist ■ Custom segment is inactive ■ The role set up to run the plug-in does not have permissions to set values on the custom segment ■ The custom segment does not impact GL		■ Enabling the Custom Segments Feature ■ Creating a Custom Segment ■ Granting a Role Permission to Manage Custom Segment Values
Transaction subsidiary {subsidiary id} is not valid for entity {entity id}. Please choose a different entity.	You entered an entity ID on the custom GL line that's not valid for the subsidiary of the transaction.	Ensure that the entity specified in setEntityId(entityId) is valid for the subsidiary of the transaction. Ensure that the entity specified in setEntityId(entityId) is valid for the subsidiary of the transaction. Multi-subsidiary customers are supported. For more information, see the help topic Enable the Multi-Subsidiary Customer Feature .
Transaction subsidiary {subsidiary} is not valid for location {location}. Please choose a different location.	You specified a location on the custom GL line that's not valid for the subsidiary of the transaction.	Specify a location in setLocationId(locationId) that's valid for the subsidiary.
Transaction subsidiary {subsidiary} is not valid for department {department}. Please choose a different department.	You specified a department that's not valid for the subsidiary.	Specify a department in setDepartmentId(departmentId) that's valid for the subsidiary.
Transaction subsidiary {subsidiary} is not valid for	You specified a class that's not valid for the subsidiary.	Specify a class in setClassId(classId) that's valid for the subsidiary.

Error Message	Cause	Preferred Action
class {class}. Please choose a different class.		
Transaction subsidiary {subsidiary} is not valid for account {account}. Please choose a different account.	You specified an account that's not valid for the subsidiary.	Specify an account in setAccountId(accountId) that's valid for the subsidiary.
Invalid custom segment value {value id} for segment {segment id}. Custom segment value does not exist or is inactive.	You specified a custom segment value that doesn't exist or is inactive.	Use a valid value ID for the custom segment.
The role restrictions for {type of segment} prevent the use of {segment}.	You specified a class, department, or location that has role restrictions.	Specify a class in setClassId(classId) , department in setDepartmentId(departmentId) , or location in setLocationId(locationId) that doesn't have role restrictions.
Account {account} cannot be used because your role restrictions are incompatible with the account's segment restriction.	You specified an account with segment restrictions that are incompatible with your role restrictions.	Specify an account in setAccountId(accountId) that has segment restrictions compatible with your role.
Entity {entity} cannot be used because your role's segment or employee restrictions are incompatible with the entity being used.	You specified an entity with segment restrictions that are incompatible with your role's segment or employee restrictions.	Specify an entity in setEntityId(entityId) that's compatible with your role's segment or employee restrictions.

Validation Error Messages for Empty Values

All of the following error messages are preceded by: **Custom GL Lines Plugin error due to a failed validation of script output:**

Error Message	Cause	Preferred Action
Account cannot be empty	You created a new CustomLine object with addNewLine() , but didn't set an account for the line with setAccountId(accountId) . All custom lines must debit or credit an account.	Make sure you set an account for every CustomLine object.
Debit/Credit cannot be empty	You created a new CustomLine object with addNewLine() , but didn't set an amount for the line with setCreditAmount(credit) or setDebitAmount(debit) . All custom lines must include a debit or credit amount.	Make sure you set an amount for every CustomLine object.
Account does not exist, id: {list of accounts not found}	You used setAccountId(accountId) to set an account for a custom line, but the account ID doesn't exist in the current account. The error message lists all account IDs used by the plug-in implementation that don't exist.	Make sure the account ID used for setAccount(accountID) exists. You can view valid account IDs in the Internal ID column on the Chart of Accounts page at Setup > Accounting > Chart of Accounts.

Error Message	Cause	Preferred Action
Class does not exist, id: {list of classes not found}	You used setClassId(classId) to set a classification for a custom line, but the class ID doesn't exist in the current account. The error message lists all class IDs used by the plug-in implementation that don't exist.	Make sure the class ID used for setClassId(classID) exists. You can view valid class IDs in the Internal ID column on the Classes page at Setup > Company > Classes.
Department does not exist, id: {list of departments not found}	You used setDepartmentId(departmentId) to set a department for a custom line, but the department ID doesn't exist in the current account. The error message lists all department IDs used by the plug-in implementation that don't exist.	Make sure the department id used for setDepartmentID(departmentID) exists. You can view valid department IDs in the Internal ID column on the Departments page at Setup > Company > Departments.
Location does not exist, id: {list of locations not found}	You used setLocationId(locationId) to set a location for a custom line, but the location ID doesn't exist in the current account. The error message lists all location IDs used by the plug-in implementation that don't exist.	Make sure the location ID used for setLocationID(locationID) exists. You can view valid location IDs in the Internal ID column on the Locations page at Setup > Company > Locations.
Entity {entity id} does not exist	You specified an invalid entity ID on the custom GL line.	Specify a valid entity in setEntityId(entityId) .

Custom Lines on Reports and in Searches

You can use the **Custom GL** flag in the Report Builder and in searches to view custom lines created by a Custom GL Lines plug-in implementation. See one of the following topics:

- [Custom GL Lines on NetSuite Reports](#)
- [Searches for Custom GL Lines](#)

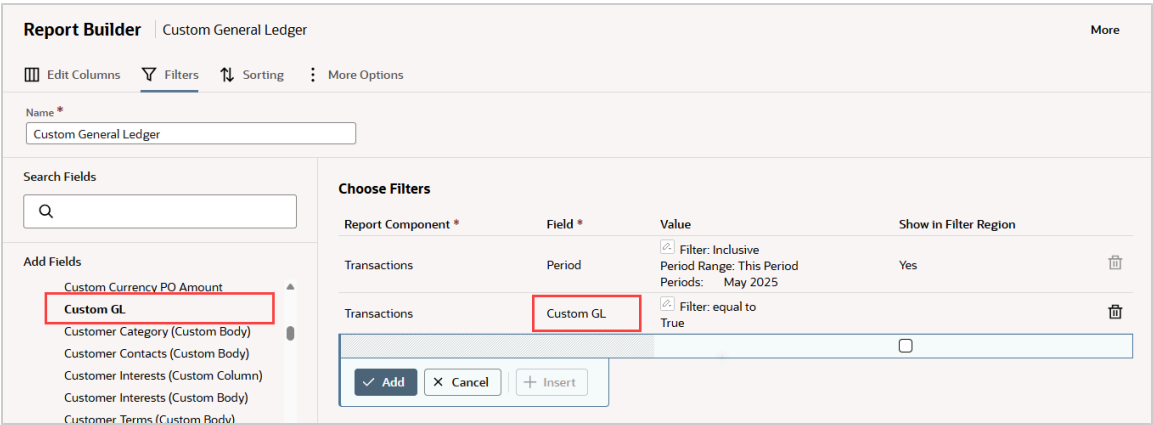
Note: You can still view custom lines on reports and search for custom lines even if you delete the plug-in implementation that created the lines. See [Deleting a Custom GL Lines Plug-in Implementation](#).

Custom GL Lines on NetSuite Reports

You can view general ledger impact for custom lines created by a Custom GL Lines plug-in implementation on NetSuite reports. In the Report Builder, you can add **Custom GL** as a column or as a filter to view report data for custom lines. For more information about customizing reports in the Report Builder, see the help topic [Report Customization](#).

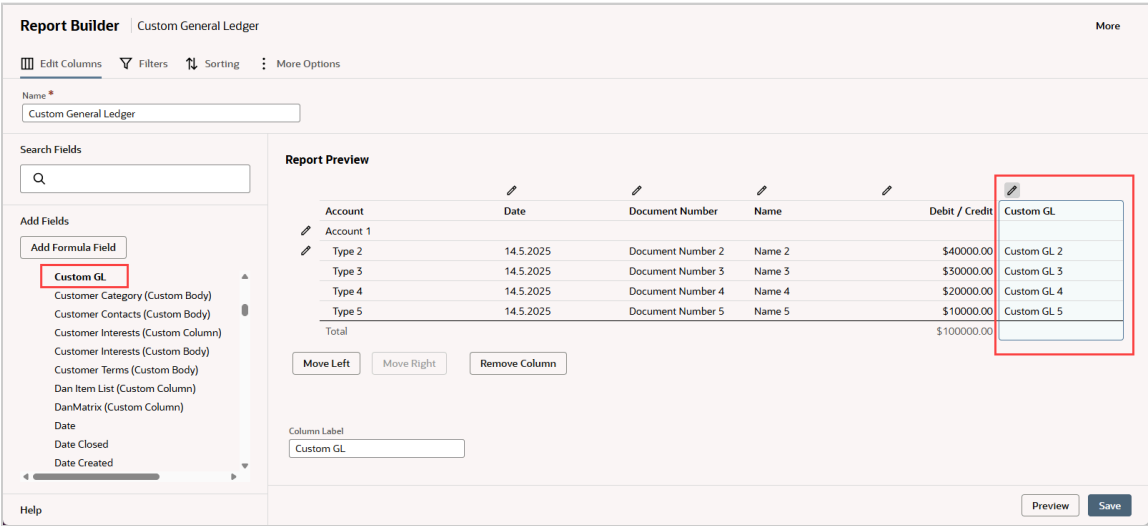
Custom GL Report Filter

The following screenshot shows the Custom GL filter in the Report Builder for a General Ledger report:



Custom GL Report Column

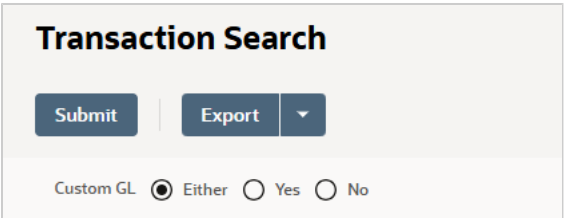
The following screenshot shows the Custom GL report column in the Report Builder for a General Ledger report:



Searches for Custom GL Lines

You can use the **Custom GL** filter on searches to search for transactions that do or do not have custom lines created by a Custom GL Lines plug-in implementation. You can use this filter on searches of type **Transaction**. You can also use this filter on searches of type **Multi-Book Accounting Transaction**, if you have the Multi-Book Accounting feature enabled.

The following screenshot shows the configuration of the Custom GL filter on a search:



For more information about using search in NetSuite, see the help topic [Simple and Advanced Searches](#).

Custom GL Lines Audit Log

The Custom GL Lines Audit Log shows custom lines created by Custom GL Lines plug-in implementations in a NetSuite account. Use the audit log to view any custom lines created for a specific transaction, all transactions, or for a specific plug-in implementation. To view the Custom GL Lines audit log page, you must have either the Administrator role or the Custom GL Lines Plug-in Audit Log permission. To view the Custom GL Lines segments audit log page, you must have either administrator or Custom GL Lines Plug-in Audit Log (Segments) permissions.

See the following topics for more information:

- [Accessing the Custom GL Lines Audit Log](#)
- [Custom GL Lines Audit Log Page](#)

Note: Custom lines remain in the audit log even if you delete the plug-in implementation that created the lines. See [Deleting a Custom GL Lines Plug-in Implementation](#).

Accessing the Custom GL Lines Audit Log

You can access the Custom GL Lines Audit Log using the following methods:

- GL Impact page. On a transaction in View mode, click Actions > GL Impact. For any custom lines on the transaction, click the name of the plug-in implementation in the **Custom Script** column:

GL Impact

Accounting Book

Primary Accounting Book

Total: 4

Accounting Book

Account

Amount (Debit)

Amount (Credit)

Base Currency

Posting

Memo

Name

Subsidiary

Department

Class

Location

Custom Script

Primary Accounting Book

221100 Bank account CZ CZK

Kč39 000,00

Czech crown

Yes

Test XYZ

Parent Company : Test Czech republic

Primary Accounting Book

2000 Accounts Payable

Kč39 000,00

Czech crown

Yes

Test XYZ

Parent Company : Test Czech republic

Primary Accounting Book

1090 Undeposited Funds

Kč1 000,00

Czech crown

Yes

Custom Line Credit

Parent Company : Test Czech republic

Custom GL test

Primary Accounting Book

1090 Undeposited Funds

Kč1 000,00

Czech crown

Yes

Custom Line Debit

Parent Company : Test Czech republic

Custom GL test

- Use a search. Create a search with the **Custom GL Lines Plug-in Audit Log** search type:

New Saved Search
Search Type
Coupon Code
Custom Employee
Customer
Customer Survey
Custom GL Lines Plug-in Audit Log
Custom Segment
Deleted Record

For more information about using search in NetSuite, see the help topic [Simple and Advanced Searches](#).



Important: You must have the Administrator role or the Custom GL Lines Plug-in Audit Log permission to view or search the Custom GL Lines audit log.

You can also view system notes for plug-in implementations. For more information, see the help topic [Viewing Plug-in Implementation System Notes](#).

Custom GL Lines Audit Log Page

The Custom GL Lines Audit Log page contains details about the custom lines the plug-in created and tried to create for a transaction. If any validations fail, they're also logged to help you determine why the script failed. Use the filters and the links in the audit log columns to get more information about the custom lines.

The following table describes the available filters for working with the audit log.

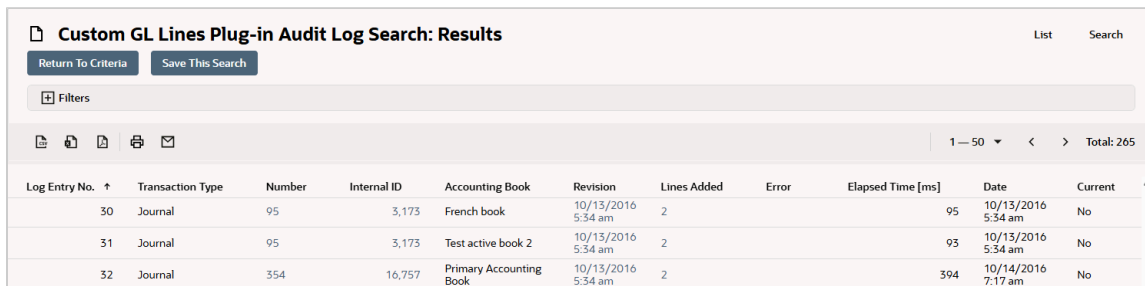
Filter Name	Description
Number	Transaction number to display.
Plug-in Implementation	Plug-in implementations and other custom script files available in the account. By default, this list includes all custom script files in the NetSuite account with script IDs that begin with customscript.
Current	Includes only custom lines that currently affect transactions in the NetSuite account. The default is Yes . Select No or All to include only custom lines for deleted transaction or lines that the plug-in implementation couldn't create due to an error.
Style	View the custom lines as a report or in a grid.

The following table describes the columns available on the default view of the Custom GL Lines audit log.

Column Name	Description
Log Entry No.	Internal ID of the custom lines in the audit log.
Transaction Type	Type of transaction on which the custom lines were created. This column shows Nonexistent if one of the following conditions are met: ■ The transaction for the custom line was deleted. ■ The custom lines couldn't be created due to an error. See the Error column for more information about the error.
Number	Record number for the transaction in NetSuite. Click the link to open the transaction record.
Lines Added	Number of custom lines that the plug-in implementation created for the transaction. Click the link to open the Custom GL Lines Audit Log Transaction Details Page .
Revision	Revision of the plug-in implementation that created the custom lines. Click the link to open the Custom GL Lines Plug-in Revision Page .

Column Name	Description
Error	Any errors that occur when NetSuite validates the plug-in implementation logic before the script runs or during the plug-in implementation script process.  Note: If you see an error in this column, the Transaction Type for the custom line is Nonexistent. If there's no error, the plug-in implementation created the custom lines successfully.
Elapsed Time (ms)	Amount of time, in milliseconds, that the plug-in implementation took to create all custom lines for the transaction.
Date	Date and time the custom lines were created.

The following screenshot shows the Custom GL Lines Audit Log page:



Custom GL Lines Plug-in Audit Log Search: Results										
Return To Criteria Save This Search Filters 1 — 50 < > Total: 265										
Log Entry No. ↑	Transaction Type	Number	Internal ID	Accounting Book	Revision	Lines Added	Error	Elapsed Time [ms]	Date	Current ▲
30	Journal	95	3,173	French book	10/13/2016 5:34 am	2		95	10/13/2016 5:34 am	No
31	Journal	95	3,173	Test active book 2	10/13/2016 5:34 am	2		93	10/13/2016 5:34 am	No
32	Journal	354	16,757	Primary Accounting Book	10/13/2016 5:34 am	2		304	10/14/2016 7:17 am	No



Tip: If you open this page from the GL impact report, you see only custom lines that affect a transaction. Use the **Current** filter to show more custom lines, for example, select **No** or **All** to see custom lines on deleted transactions.

The Custom GL Lines Audit Log Segments page shows the segments set by a specific transaction. For information, see [Custom GL Lines Audit Log Segments Page](#).

You can also view system notes for plug-in implementations. For information, see the help topic [Viewing Plug-in Implementation System Notes](#).

Custom GL Lines Audit Log Transaction Details Page

The Custom GL Lines Audit Log Transaction Details page shows the balancing custom lines added to a specific transaction. Click the link in the **Lines Added** column on the [Custom GL Lines Audit Log Page](#) to access the Custom GL Lines Audit Log Transaction Details page.

The following table describes the columns available on the default view of the Custom GL Lines Audit Log Transaction Details page:

Column	Description
Log Entry No.	Internal ID of the custom lines in the audit log. This is the same ID as Log Entry No. on the Custom GL Lines Audit Log Page . Click the link to go to the audit log page for the custom line.
Account ID	Internal ID of the account that the custom line credited or debited. This ID appears on the Chart of Accounts page at Setup > Accounting > Chart of Accounts.

Column	Description
Credit *	Amount of credit for the custom line. This value is set either by the plug-in implementation itself when it maps custom lines to secondary books, if applicable, or with the setCreditAmount(credit) method.
Debit *	Amount of debit for the custom line. This value is set either by the plug-in implementation itself when it maps custom lines to secondary books, if applicable, or with the setDebitAmount(debit) method.*
Memo	Text added to the Memo field on the custom line. This value is set with the setMemo(memo) method by the plug-in implementation.
Class ID	Class ID for the custom line set with the setClassId(classId) method by the plug-in implementation. For more information about classes in NetSuite, see the help topic Departments and Classes Overview .
Department ID	Department ID for the custom line set with the setDepartmentId(departmentId) method by the plug-in implementation. For more information about departments in NetSuite, see the help topic Departments and Classes Overview .
Location ID	Location ID for the custom line set with the setLocationId(locationId) method by the plug-in implementation. For more information about locations in NetSuite, see the help topic Locations Overview .
Total Segments Set	Number of custom segments that were set by the plug-in implementation.
Entity ID	Entity ID for the custom line set with the setEntityId(entityId) method by the plug-in implementation.
* The currency for the amount depends on the transaction currency and the base currency for the accounting book and subsidiary. See Multiple Currencies Examples .	

The following screenshot shows the Custom GL Lines Audit Log Transaction Details page:

Custom GL Lines Audit Log - Transaction Details										
View Custom GL Lines A...n Details Default Customize View										List Search
Filters										
										Total: 2
Log Entry No. ↓	Account ID	Debit	Credit	Memo	Class ID	Department ID	Location ID	Total Segments Set	Entity ID	Book Specific
17	1	0	1000	Custom Line Credit				2		No
17	2	1000	0	Custom Line Debit				2		No

Custom GL Lines Audit Log Segments Page

The Custom GL Lines Audit Log Segments page shows the segments that were set by a specific transaction. Click the link in the **Total Segments Set** column of the Custom GL Lines Audit Log Transaction Details page to access the Custom GL Lines Audit Log Segments page. Administrator or Custom GL Lines Plug-in Audit Log (Segments) permissions are required to view the Custom GL Lines Audit Log Segments page.

The following table describes the columns available on the default view of the Custom GL Lines Audit Log Segments page:

Column	Description
Log Entry No.	Internal ID of the custom line in the audit log. This is the same ID as the Log Entry No. column on the Custom GL Lines Audit Log Page . Click the link to go to the audit log page for the custom line.
Line	Line number of the custom GL line.
Segment ID	ID for the custom segment definition.
Segment	Description of the custom segment definition.
Segment Value ID	ID of the segment value for the custom line set with the <code>setSegmentValueId(segmentId, segmentValueId)</code> method by the plug-in implementation. For more information, see setSegmentValueId(segmentId, segmentValueId) .
Segment Value	Custom segment value.

The following screenshot shows the Custom GL Lines Audit Log Segments page:

Custom GL Lines Audit Log - Segments					
View: Custom GL Lines ...Segments Default Customize View					
Filters					
Total: 2					
Log Entry No. ↓	Line	Segment ID	Segment	Segment Value ID	Segment Value
9	1	CSEG1	Rev Rec test XX		
9	1	CSEG2	Product line		

Custom GL Lines Plug-in Revision Page

The Custom GL Lines Plug-in Revision page shows the specific version of the plug-in implementation and associated library files that created a custom line. Use this page to determine the versions of the plug-in implementation script file and library file used to create the custom lines.

Click the link in the **Revision** column on the [Custom GL Lines Audit Log Page](#) to access this page.

The following table describes the columns available on the default view of the Custom GL Lines Plug-in Revision page:

Column	Description
Plug-in Implementation	Name of the plug-in implementation that created the custom lines.
File Name	File name of the plug-in implementation script file. For more information about this file, see Plug-in Implementation Script File Creation for SuiteScript 1.0 .
File Contents	Snapshot of the plug-in implementation script file contents. Note: This information comes from the NetSuite database and is for reference only. You cannot click (more...) to see the rest of the file contents. To view the complete file content, open the script record for the plug-in implementation script file.
Modified	Date and time the plug-in implementation script file was last modified.

Column	Description
Modified By	User account that last modified the plug-in implementation script file.
Libraries	Associated library file names for the plug-in implementation that were specified during creation of the plug-in implementation. All library files are listed. For more information, see Adding the Plug-in Implementation .
Libraries Modified	Latest date and time when any one of the libraries for the plug-in implementation was last modified.

The following screenshot shows the Custom GL Lines Plug-in Revision page:

Custom GL Lines Plug-in Revisions							List	Search
View: Custom GL Lines P... Revision Default Customize View								
Filters								
							Total: 1	
Plug-in Implementation	File Name	File Contents	Modified	Modified By	Libraries	Libraries Modified		
Custom GL test	Custom GL test	Download	7/27/2016 5:47 am					

Reviewing Custom GL Lines Plug-in Executions

Custom GL line plug-ins may run asynchronously in the following situations:

- If you have several custom GL line plug-in implementations that slow down transaction save times.
- If you use both the Custom GL Lines Plug-in and Inventory features, and the cost of goods sold (COGS) changes, cost amounts are updated. The Custom GL Lines Plug-in runs asynchronously to update the GL impact in the transactions.

When the COGS process updates a transaction, the GL impact is updated as well. Then, the Custom GL Lines Plug-in implementations run on the transaction. If a change triggers a COGS update, the Custom GL Lines Plug-in process runs again to update the GL impact. When you view the GL Impact page, you may see a message at the top of the page to indicate that the GL impact may change. This message appears if an update process is scheduled or in progress.

To address any exceptions that occurred when a custom GL line plug-in ran asynchronously, the Review Custom GL Plug-in Executions page displays Incomplete and Failed executions. You must have the Administrator role or the Review Custom GL Plug-ins Executions permission to access the page. Go to Customization > Plug-ins > Review Custom GL Plug-in Executions.

Review Custom GL Plug-in Executions						More
Execute selected Ignore selected Refresh						
Accounting Period: - All - Include closed periods						Status: Incomplete
Incomplete 0 Failed 0						
Mark All Unmark All						
Transaction ID	Plug-in Implementation	Execution Date	Transaction Type	Accounting Period	Error Message	
991	Update GL for Petty Cash	6/12/2017 11:47 am	Journal	Jun 2017	Method customizeGlimpact of plug-in CustomGILinesPlugin not implemented.	
Execute selected Ignore selected Refresh						

Transactions appear on the Review Custom GL Plug-in Executions page only if the last run of the custom GL plug-in was incomplete or failed. The page displays incomplete and failed Custom GL Plug-in executions from subsidiaries and accounting books to which you have access.

By default, failed or incomplete transactions for all accounting periods are displayed. Select the accounting period as required. To include closed periods in the Accounting Period list, check the Include Closed Periods box.

Each task displays the transaction ID, transaction type, accounting period, and run date. An incomplete task has at least one pending or running task for the custom GL plug-in in the selected period.

Note: If you update a transaction in a closed period and it triggers the Custom GL Lines Plug-in, an error appears. You can either ignore the error or open the period to run the plug-in again.

For any failed executions, you can select the item and click Execute Selected to run the plug-in again. The page refreshes, the status changes to incomplete, and the line appears on the Incomplete subtab.

You can ignore an error, but if you want to run the custom GL lines plug-in in the future, you have to edit the transactions manually. To ignore an error, check the line and click Ignore Selected. A confirmation message appears. Click Yes to confirm and remove the transaction from the list.

To view information about custom GL line plug-in executions that completed successfully, see the [Custom GL Lines Audit Log Page](#).

Deleting a Custom GL Lines Plug-in Implementation

If you delete a Custom GL Lines plug-in implementation or uninstall a bundle that contains a plug-in implementation, here's what happens in your the NetSuite account:

- Custom lines created by the plug-in implementation remain in the NetSuite database. The custom lines remain linked to the original transaction and appear in the GL impact for the transactions on which they were created. The custom lines also remain on any reports that list the GL impact of the transaction, and you can search for the custom lines. See [Custom Lines on Reports and in Searches](#).
- The custom lines remain in the Custom GL Lines audit log. See [Custom GL Lines Audit Log](#).

Note: If you uninstall a bundle with a plug-in implementation, the script file and any library files are also removed.

To delete a Custom GL Lines plug-in implementation:

1. Go to Customization > Plug-ins > Manage Plug-ins.
2. On the Manage Plug-ins page, disable the implementation you want to delete and click **Save**.
3. Go to Customization > Plug-ins > Plug-in Implementations.
4. On the Plug-in Implementations page, click **Edit** next to the plug-in implementation.
5. On the Plug-in Implementation page, click Actions > Delete. Click **OK** to confirm the deletion.